

图论及其应用

第一章：图的基本概念

主讲人：潘嵘

2026-03-02

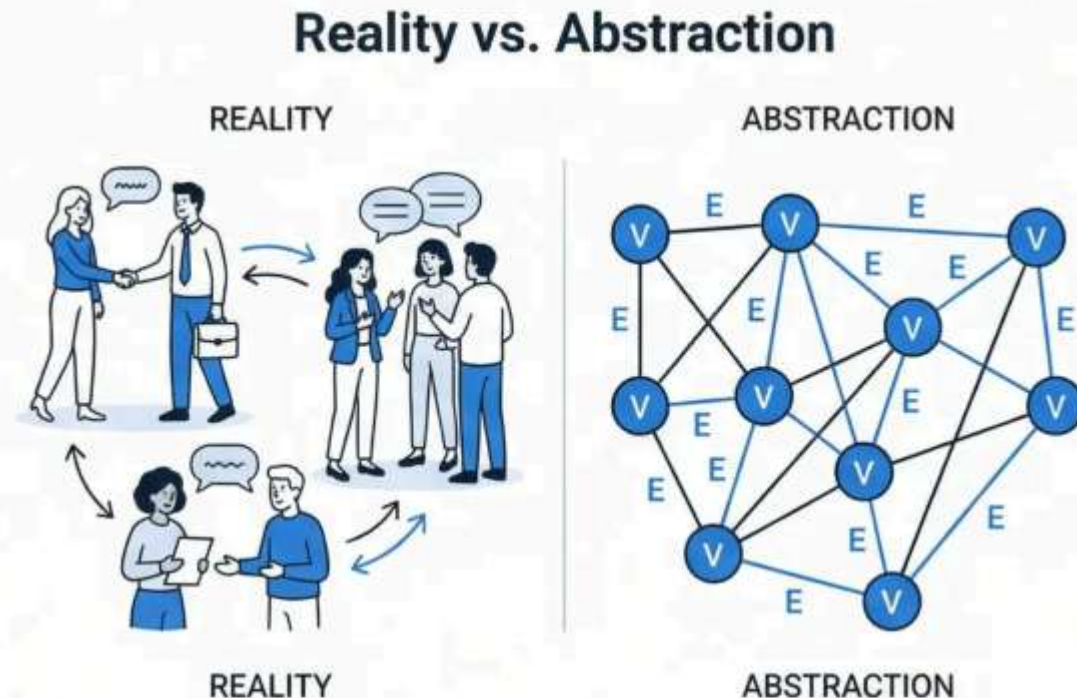


万物皆可为图：抽象建模的本质

图论，并非研究函数图象或摄影图片，而是致力于将现实世界中的“事物”与其“相互联系”进行抽象建模。

图论研究的核心：

- **事物 (顶点 V)**：任何可以被独立识别的实体
- **相互联系 (边 E)**：事物之间存在的某种关系

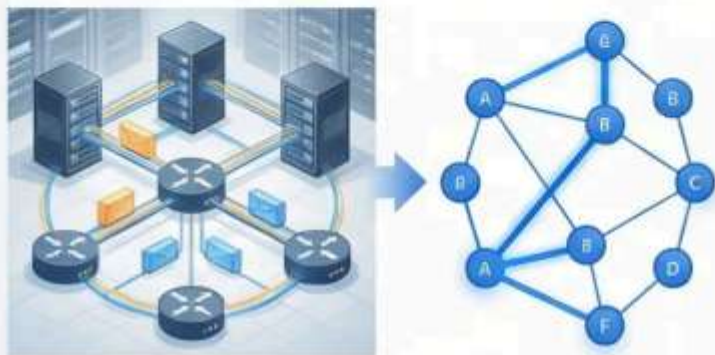


这种抽象能力是计算机科学、网络安全等领域解决复杂问题的基石。

图论的广泛应用领域

计算机网络

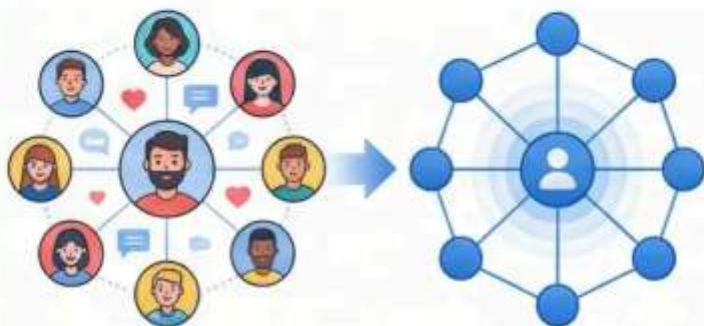
- 路由器作为顶点
- 网络链路作为边
- 路径优化与拓扑分析



Internet infrastructure modeling

社交网络

- 用户作为顶点
- 朋友关系作为边
- 影响力传播分析



Social media platform analysis

化学分子结构

- 原子作为顶点
- 化学键作为边
- 分子性质预测



Drug discovery and materials science

图论：将复杂世界抽象为顶点与边的通用语言

回溯起源：哥尼斯堡七桥问题 (Euler, 1736)

时代背景

18世纪的普鲁士城市哥尼斯堡

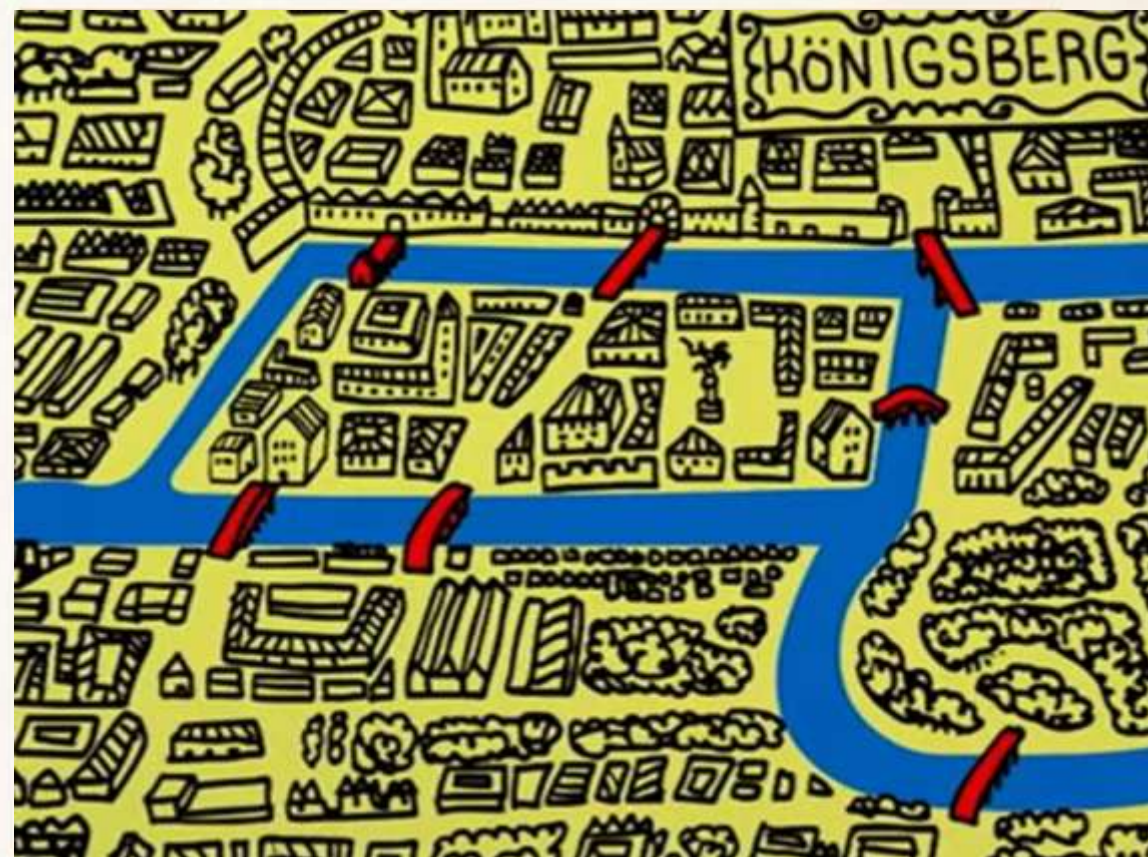
普雷格尔河环绕两座岛屿，将城市分割成四块陆地

七座桥梁连接各个区域

核心挑战

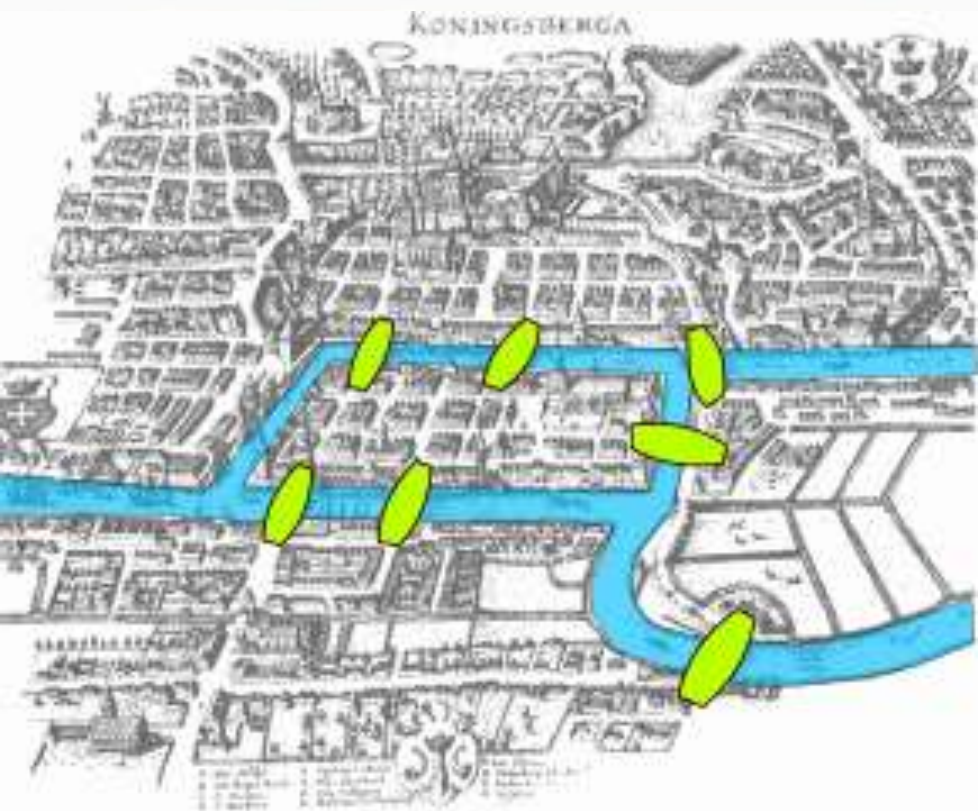
市民们的日常疑问：能否一次性走过每座桥且每座桥只走一次？

这个看似简单的问题困扰了许多人

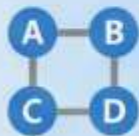


Euler 的天才简化：从现实到模型

欧拉 (Euler) 的天才之处在于，他没有陷入对地理细节的纠结，而是将问题抽象化，剥离了所有非本质的元素。他意识到，问题的关键不在于陆地的形状或桥梁的长度，而在于它们之间的连接关系。



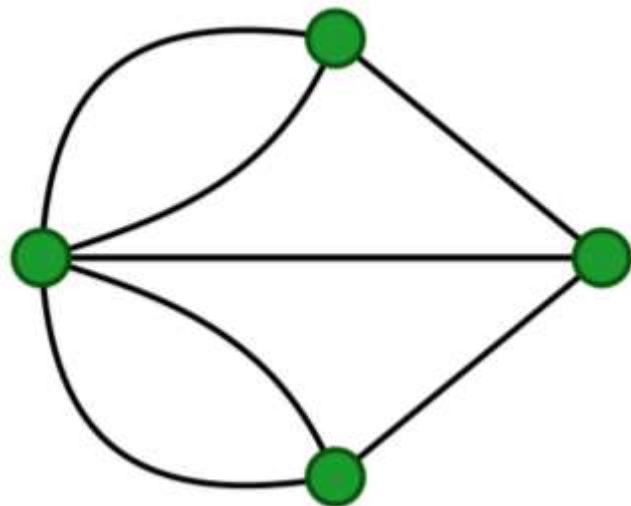
Euler 的天才简化：



陆地 → 顶点 (Vertex)



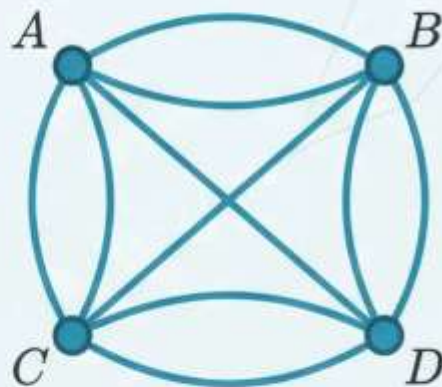
桥梁 → 边 (Edge)



Key Insight: 通过这种抽象，一个真实的地理难题，被转化成了一个纯粹的数学问题。

图论的起点与数学力量

经过欧拉的抽象，哥尼斯堡七桥问题被转化为一个关于图的遍历问题。通过对这个抽象模型的分析，欧拉不仅解决了这个特定问题（证明无法实现一笔画），更重要的是，他开创了一个全新的数学分支——图论。



结论：

- 这就是图论的起点。
- 它展示了数学将复杂现实问题简化、抽象，并最终解决的强大力量。
- 这一思想在现代科学与工程领域无处不在，是离散数学的核心组成部分。

从一个看似“无聊”的问题，诞生了影响深远的数学理论。

图论之光：从理论基石到大国重器

图论是计算机科学、软件工程、网络安全、保密等专业的核心地基

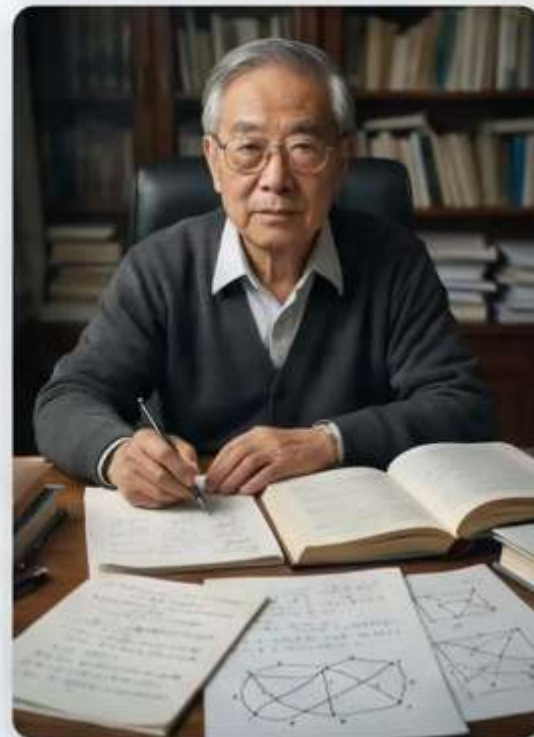
我们的使命与责任：

- **学科基础是核心**：图论是构建复杂算法、设计高效网络、保障系统安全的基础
- **勇攀科学高峰**：做有担当的未来工程师
- **解决“卡脖子”问题**：只有深入理解和掌握核心基础理论，才能在关键领域实现创新突破，不再受制于人

让我们向中国数学家管梅谷先生致敬

他提出的“**中国邮路问题**”（又称中国邮递员问题）及“**破圈法**”，在图论应用领域做出了卓越贡献，为我们树立了典范

这是将理论与实践结合，服务国家需求的生动例证

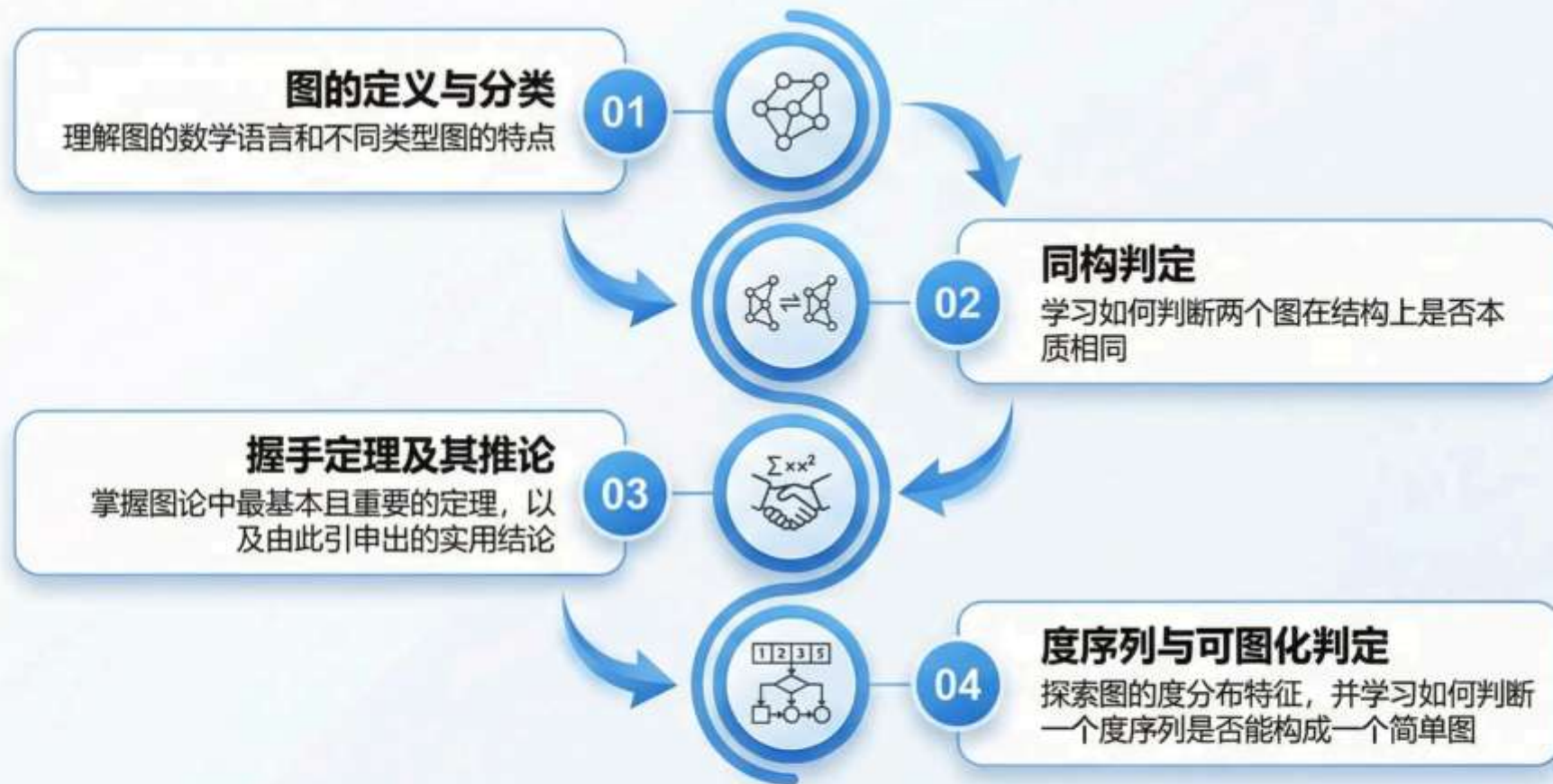


Chinese Postman Problem

Cycle-Breaking Method

第一章：本章学习路线图

为了更好地掌握图论的基本概念，本章我们将沿着以下路线进行深入学习：



每个模块都**环环相扣**，旨在构建您对**图论**扎实而全面的理解。

定义 1：图的数学刻画

Definition 1 (Graph): 一个图 G 定义为一个有序对 (V, E) , 记为 $G=(V, E)$.

- $V(G)$: 非空集合, 称为**顶点集** (Vertex set)
 - 其元素是图中的节点或点
 - 表示系统中的实体或对象
- $E(G)$: 由 V 中的点组成的无序点对构成的集合, 称为**边集** (Edge set)
 - 其元素是连接顶点的边
 - 表示实体之间的关系或连接

示例图 $G=(V, E)$:

$$V(G) = \{v_1, v_2, v_3, v_4, v_5\}$$

$$E(G) = \{e_1, e_2, e_3, e_4, e_5, e_6\}$$

$$e_1 = (v_1, v_2)$$

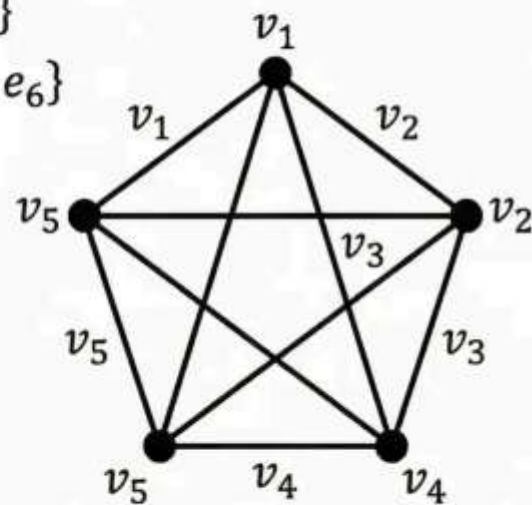
$$e_2 = (v_2, v_3)$$

$$e_3 = (v_3, v_4)$$

$$e_4 = (v_4, v_5)$$

$$e_5 = (v_5, v_1)$$

$$e_6 = (v_1, v_3)$$



基本术语：阶数 (Order) 与边数 (Size)

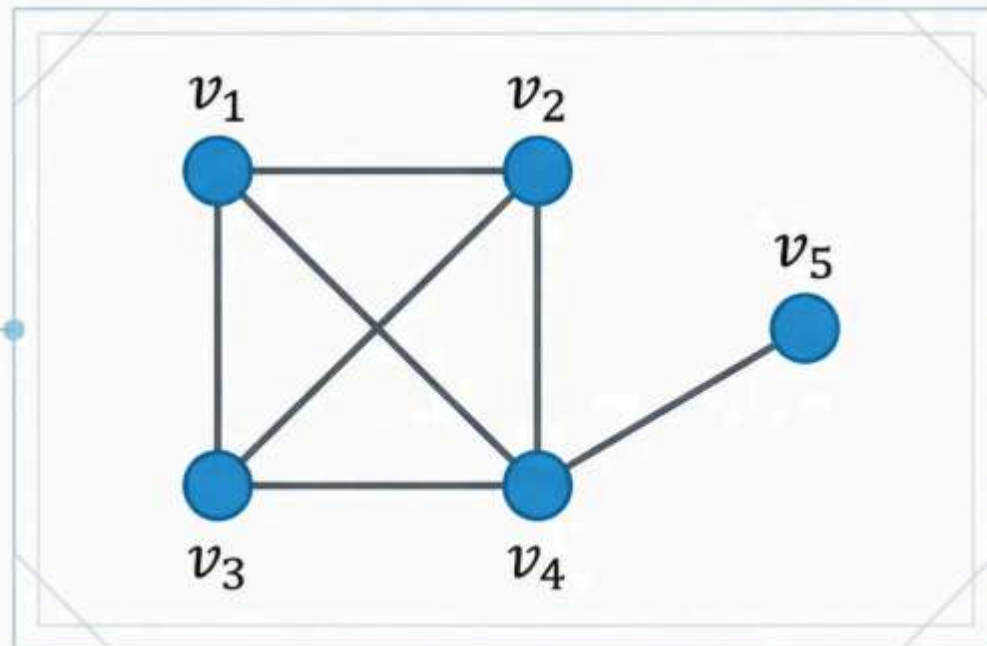
阶数 (Order) 定义

$n(G) = |V|$ 顶点的个数

边数 (Size) 定义

$m(G) = |E|$ 边的个数

性质说明：图的静态属性



实例计算：

对于图 G: $n(G) = 5$ (顶点个数)

$m(G) = 7$ (边的个数)

基本术语：相邻 (Adjacent) 与关联 (Incident)

相邻 (Adjacent)

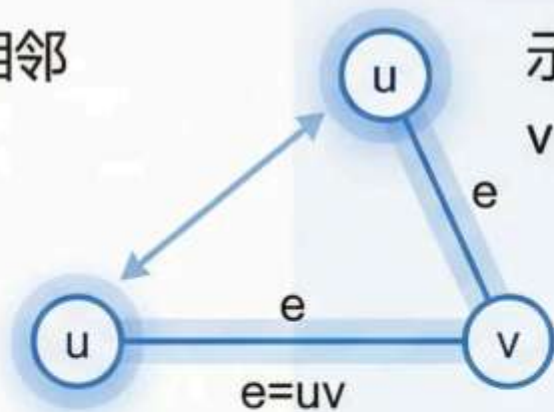
若边 e 连接顶点 u 和 v (记作 $e=uv$)，则称顶点 u 和 v 相互**相邻**

示例：在边 uv 中， u 和 v 互相相邻

关联 (Incident)

当顶点 u 是边 e 的一个端点时，称顶点 u 与边 e 相互**关联**

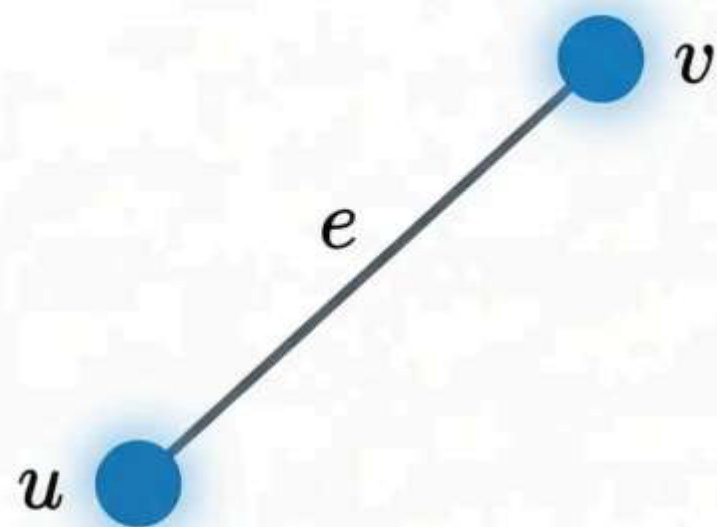
示例：顶点 u 与边 uv 关联，顶点 v 也与边 uv 关联



基本术语：端点 (End vertices)

对于一条边 $e = uv$, 顶点 u 和 v 被称为这条边 e 的端点

边是连接两个顶点的逻辑纽带



示例：在边 (v_1, v_2) 中, v_1 和 v_2 互为端点

基本术语：相邻的边 (Adjacent Edges)

相邻的边 (Adjacent Edges)：如果两条边共享一个公共的端点，则称这两条边相邻。

设边 $e_1 = uv$ 和边 $e_2 = vw$ ，由于它们在顶点 v 处相遇，因此 e_1 和 e_2 是相邻的边。

 **关键理解：** 相邻的边通过共同的顶点建立连接关系，这是图拓扑结构的重要特征。

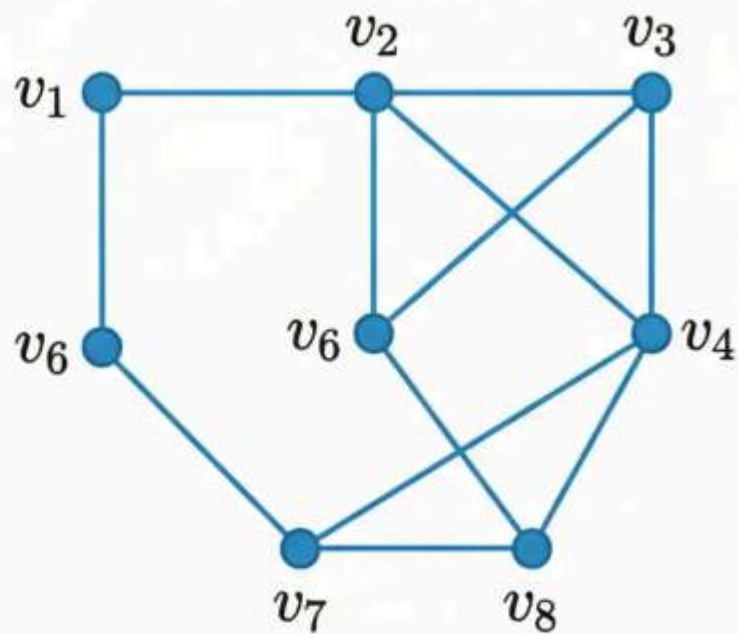


基本术语：有限图的概念

有限图 (Finite Graph)：如果图 G 的顶点集 $V(G)$ 和边集 $E(G)$ 都是有限集合，则称 G 为**有限图**。

- **特点**：绝大多数实际应用中涉及的图都是有限图
- **示例**：任何实际网络（如互联网、社交网络）的局部都可以被视为有限图

重要说明：本课程主要讨论有限图



基本术语：平凡图与空图

平凡图 (Trivial Graph)

只有一个顶点而没有边的图

$$V = \{v_1\}, E = \emptyset$$

- 最简单的图结构
- 表示孤立的个体



v_1

一个孤立的个体

空图 (Null Graph)

拥有一个或多个顶点，但边集为空的图

$$V = \{v_1, v_2, v_3\}, E = \emptyset$$

- 所有顶点都是孤立点
- 表示互不认识的群体



v_1



v_2



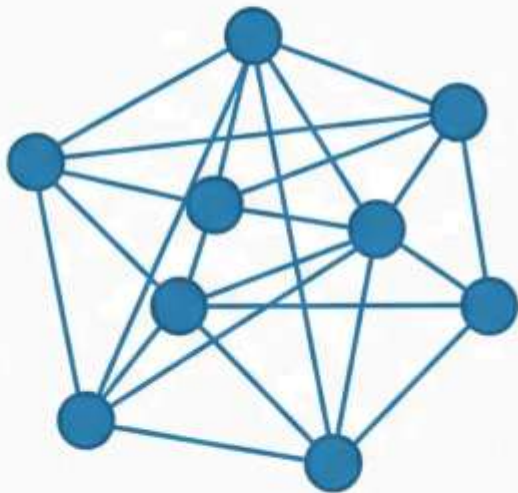
v_3

一群互不认识的人

特殊形态：有限图与平凡图对比

有限图 (Finite Graph)

- **定义：** $V(G)$ 和 $E(G)$ 均为有限集合的图
- **特点：** 绝大多数实际应用中涉及的图都是有限图
- **示例：** 任何实际网络（如互联网、社交网络）的局部都可以被视为有限图



平凡图 (Trivial Graph)

- **定义：** 仅包含一个顶点，且不含任何边的图
- **特点：** 是最简单的图结构
- **示例：** 一个孤立的个体，没有任何联系



特殊形态：空图与环 (Loop)

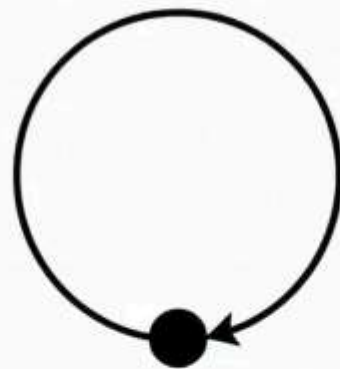
空图 (Null Graph)

- **定义：**拥有一个或多个顶点，但边集 E 为空的图
- **特点：**所有顶点都是孤立点
- **示例：**一群互不认识的人



环 (Loop)

- **定义：**一条边，其两个端点重合为同一个顶点
- **特点：**表示事物与自身的某种关系
- **示例：**自我指涉的系统，任务的自循环依赖

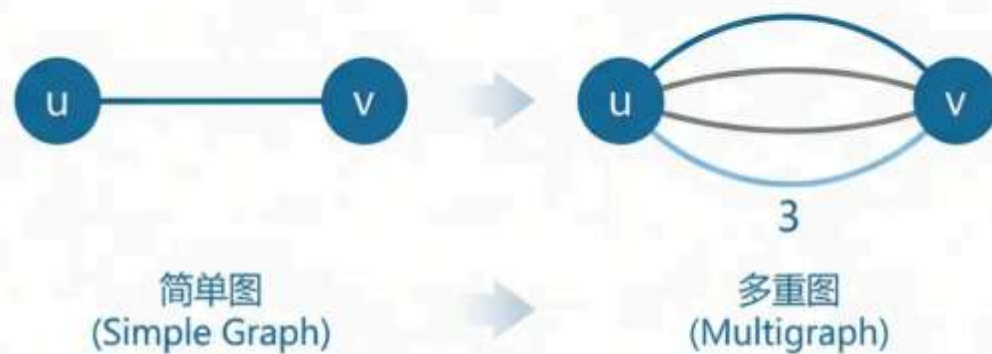


特殊形态：重边 (Multiple edges)

重边 (Multiple edges) 定义

连接相同一对顶点的多条边

- 特点
 - 两个实体间存在多种类型关系或多条独立连接
 - 边的重数 (multiplicity) : 连接同一对顶点的边的数量
- 实施应用
 - 交通网络：两地间多条不同路线 (公路、铁路、航线)
 - 社交关系：用户间多种关系类型 (朋友、同事、家人)
 - 通信网络：设备间多条独立链路
- 多重图 (Multigraph) : 包含重边的图
 - 区别于简单的图，是基础发了复杂的网络格模型



特殊形态：形态总结矩阵

特殊形态	定义	示例	建模意义
有限图 (Finite Graph)	顶点集和边集均为有限集合	任何实际网络的局部	绝大多数实际应用场景
平凡图 (Trivial Graph)	只有一个顶点且无边	孤立的个体	独立系统或起始状态
空图 (Null Graph)	有顶点但边集为空	一群互不认识的人	无连接关系的集合
环 (Loop)	边的两端点重合	自我指涉的系统	自循环依赖或反馈
重边 (Multiple Edges)	连接相同一对顶点的多条边	两地间多条交通路线	多重关系或并行连接



特殊形态：复合图的构成

复合图 (Complex Graph) : 含有环或重边的图

- 包含环 (Loop) 的图 → 复合图
- 包含重边 (Multiple Edges) 的图 → 复合图
- 同时包含环和重边的图 → 复合图

统一分类：为后续研究提供清晰的理论基础



含环的复合图



含重边的复合图



环与重边并存

简单图 (Simple Graph) 的严谨定义

Definition (Simple Graph)

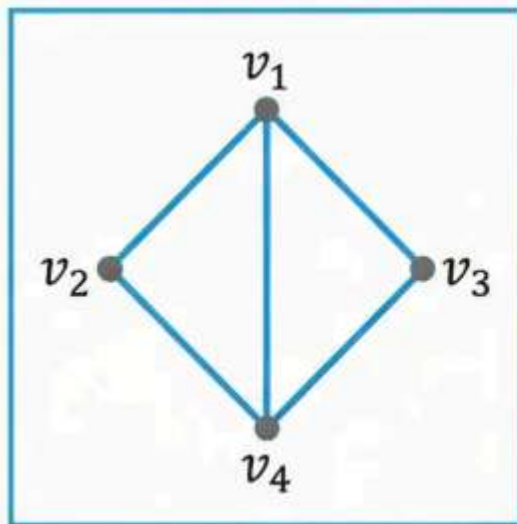
简单图是既没有环 (loops) 也没有重边 (multiple edges) 的图

记作: $G = (V, E)$, 其中每对顶点之间至多有一条边

关键特征

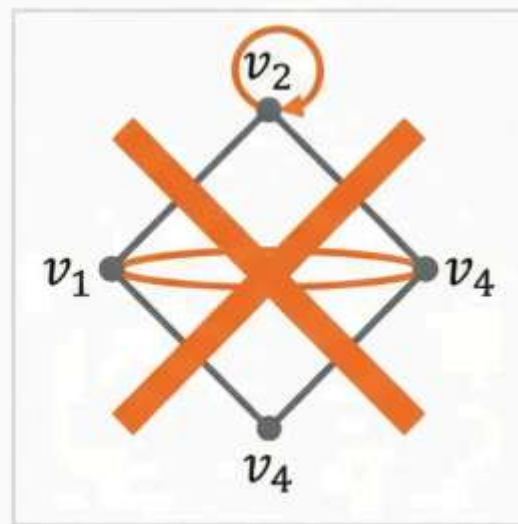
- ✓ 无自环: 没有顶点连接自身的边
- ✓ 无重边: 任意两顶点间最多一条边
- ✓ 无向图: 边没有方向性

简单图示例



✓ 简单图: 符合定义

非简单图对比



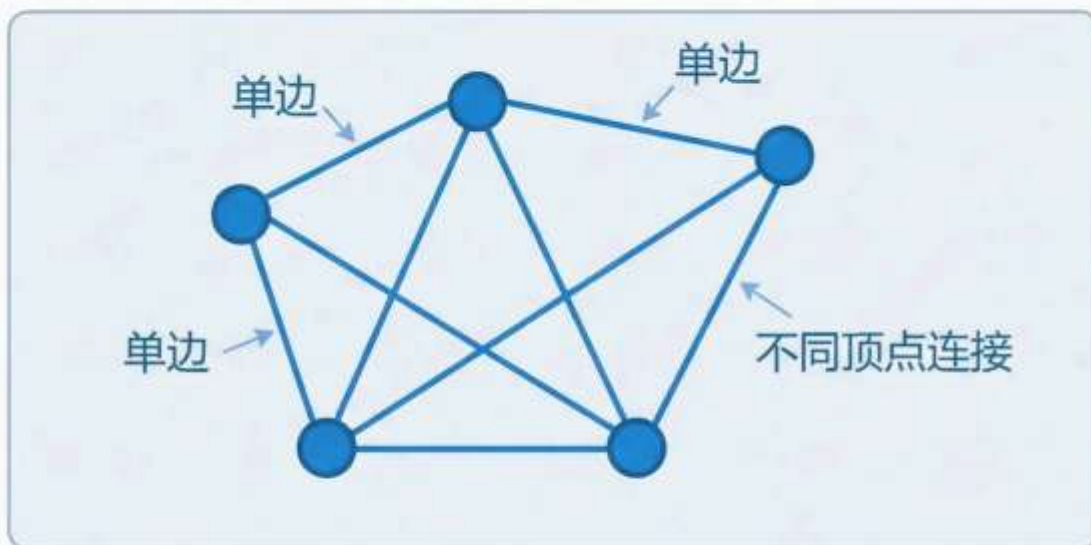
✗ 复合图: 含环和重边

重要性说明的要性说明: 简单图是图论研究的基础对象
本课程后续定理和算法主要针对简单图
除非特别说明, 默认讨论简单图

简单图与复合图的视觉区分

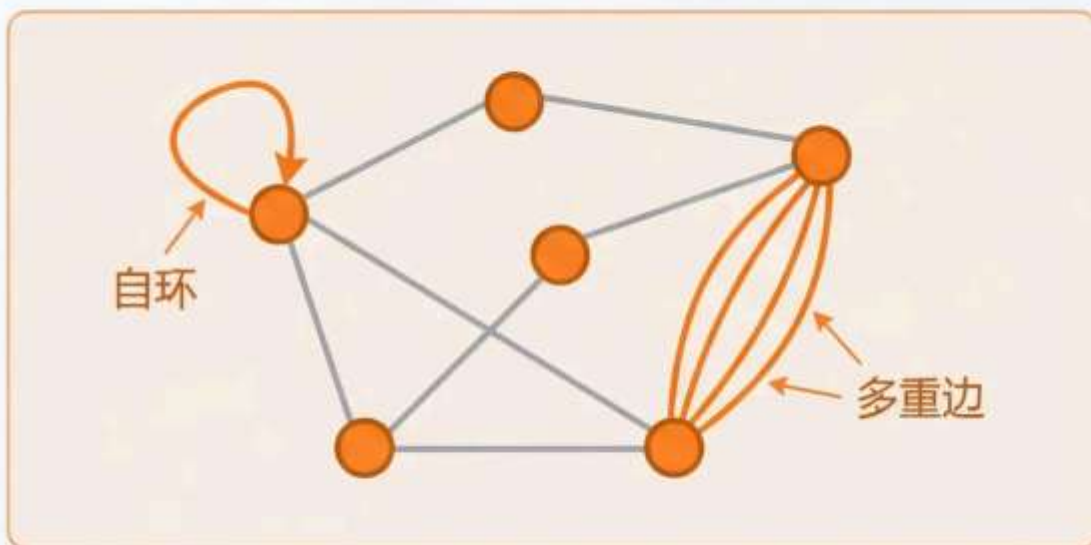
简单图 (Simple Graph)

- ✓ 无环 (No loops)
- ✓ 无重边 (No multiple edges)



复合图 (Complex Graph)

- ⚠ 包含环 (Contains loops)
- ⚠ 包含重边 (Contains multiple edges)



识别要点: 检查是否存在自环和平行边

交互判定：谁是简单图？

结合图例进行交互式判定练习

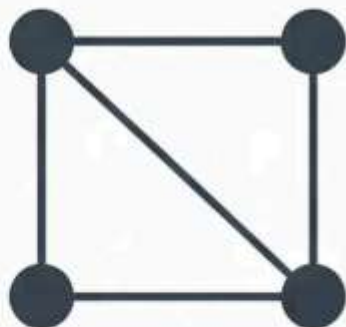


图 A ☐

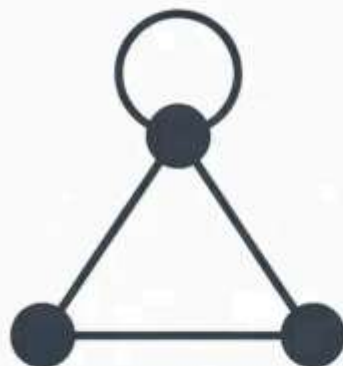


图 B ☐



图 C ☐



图 D ☐

请选择哪些是简单图

提示：简单图既无环也无重边

课程默认约定：简单图

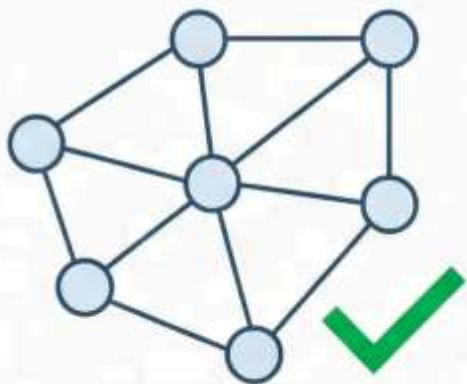


本课程若无特殊说明，默认讨论对象均为简单图

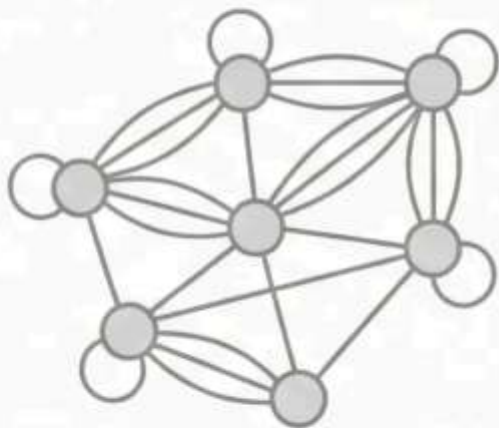
简化理论推导：避免环和重边的特殊情况讨论

聚焦核心结构：强调顶点间的基本连接关系

便于算法分析：大多数经典算法基于简单图设计



默认研究对象



特殊说明时

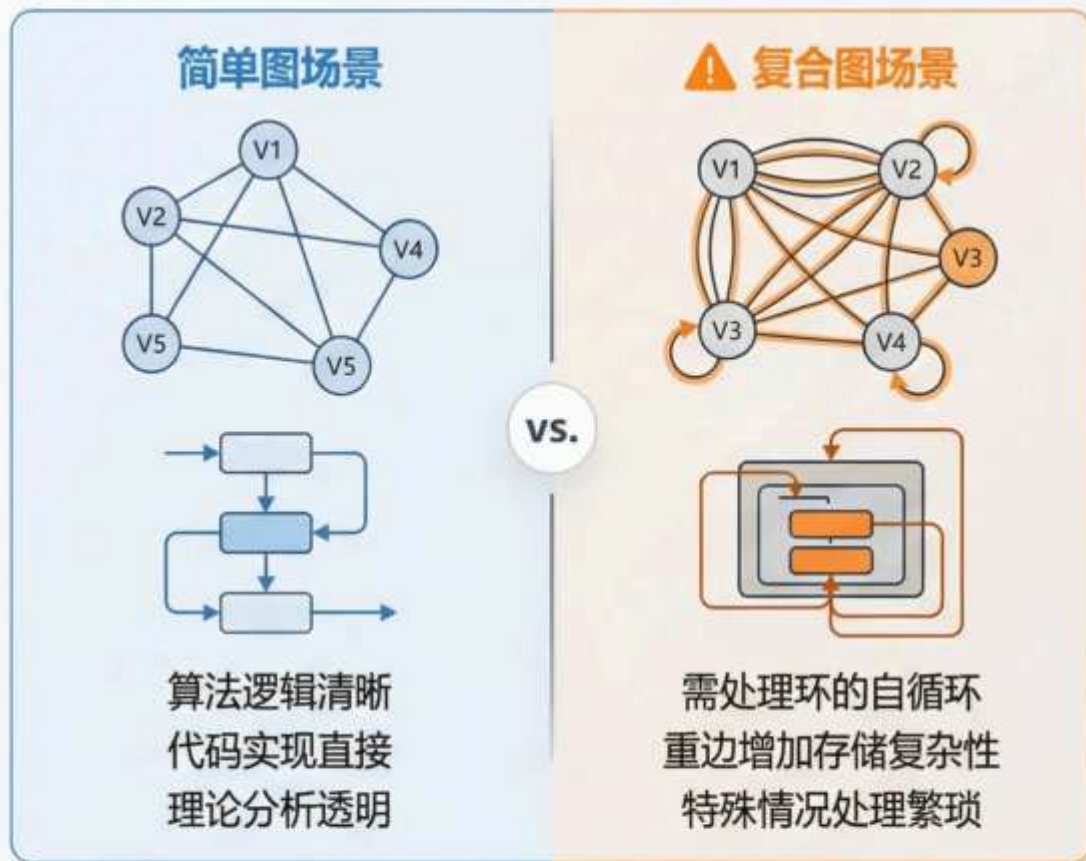
- ✓ 理论推导更加简洁
- ✓ 算法设计更易理解
- ✓ 性质证明更加直观

简单图的建模优势

为什么优先研究简单图？

- 1. 算法设计**简化** (Algorithm Simplification)
 - 避免环和重边的特殊处理逻辑
 - 减少边界条件检查, 提升算法效率
 - 经典算法 (最短路径、生成树) 最初针对简单图设计
- 2. 聚焦**核心拓扑结构** (Focus on Core Topology)
 - 强调顶点间的基本连接关系
 - 突出图的本质结构特性
 - 便于理论推导和性质证明
- 3. 数学**分析优势** (Mathematical Analysis Benefits)
 - 度数计算直观明确
 - 矩阵表示更加规整
 - 理论定理表述简洁

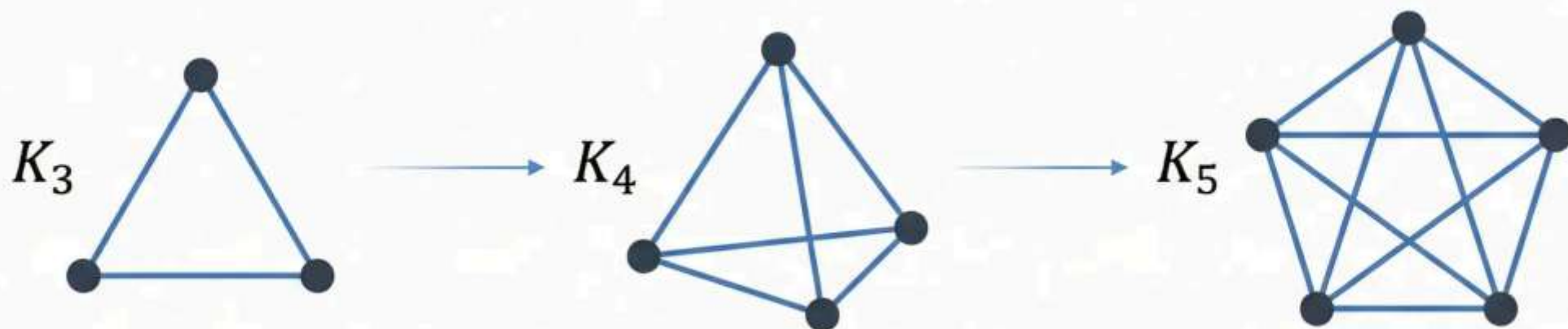
复杂度对比示意



 **实际应用说明:** 当需要处理环和重边时, 图论提供专门的扩展理论和算法。**简单图作为基础**, 为理解复杂图奠定坚实根基。

完全图 (Complete Graph) K_n

完全图：每两个不同顶点之间都有一条边相连的简单图



- n 个顶点的完全图记为 K_n
- 体现图中“极致连接”的概念
- 在同构意义下, n 个顶点的完全图只有一种

完全图 K_n 具有最多的边数

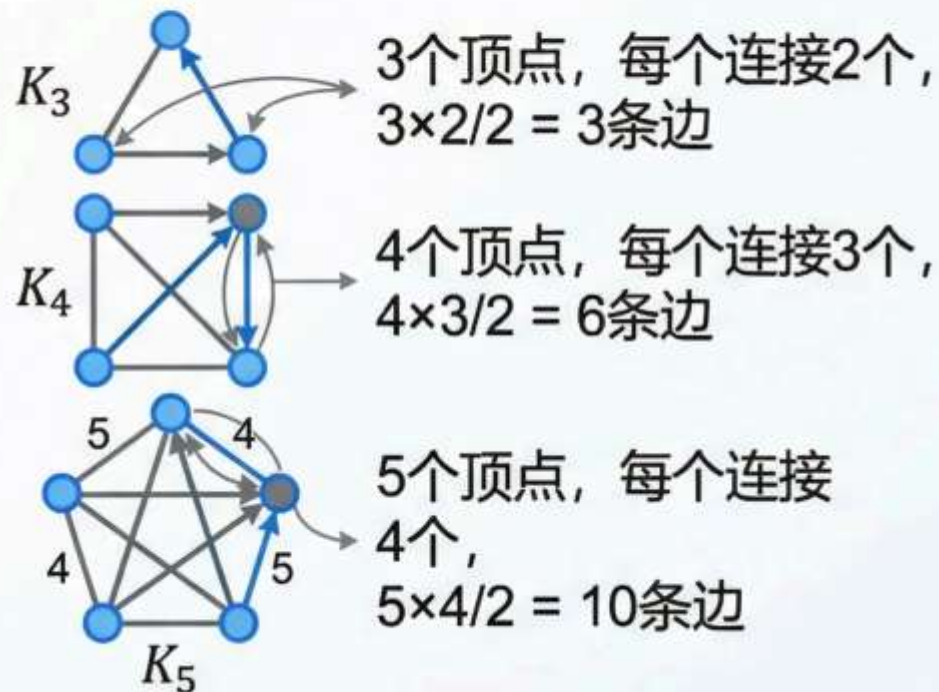
完全图的边数性质

为什么 K_n 恰好有 $\frac{n(n-1)}{2}$ 条边?

1. **组合逻辑**: 在 n 个顶点中选择 2 个顶点形成一条边
2. **数学表达**: 边数 = $\binom{n}{2} = \frac{n!}{2!(n-2)!} = \frac{n(n-1)}{2}$
3. **直观理解**: 每个顶点连接其余 $n-1$ 个顶点, 但每条边被计算两次

核心公式

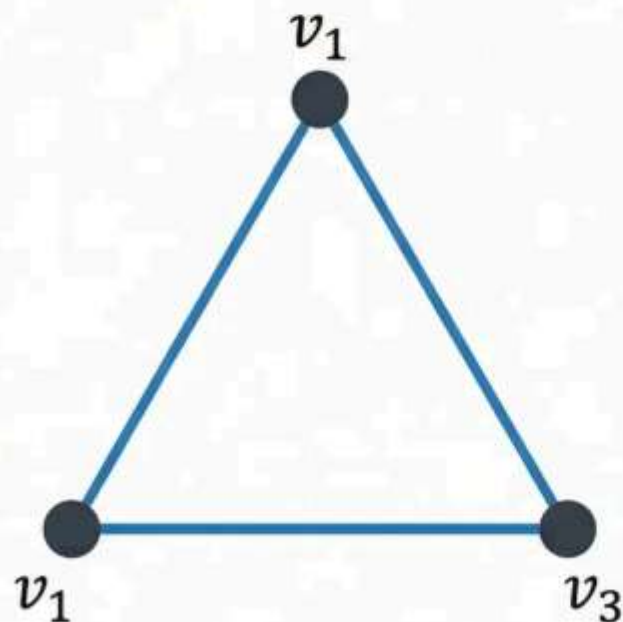
$$m(K_n) = \frac{n(n-1)}{2}$$



Graph	Vertices (n)	Edges (m)	Formula Check
K_3	3	3	$\frac{3 \times 2}{2} = 3$ ✓
K_4	4	6	$\frac{4 \times 3}{2} = 6$ ✓
K_5	5	10	$\frac{5 \times 4}{2} = 10$ ✓

- **重要意义**: “ K_n 是 n 阶简单图中边数的**上限**”
- **实际含义**: 任何 n 阶简单图的边数 $m \leq \frac{n(n-1)}{2}$

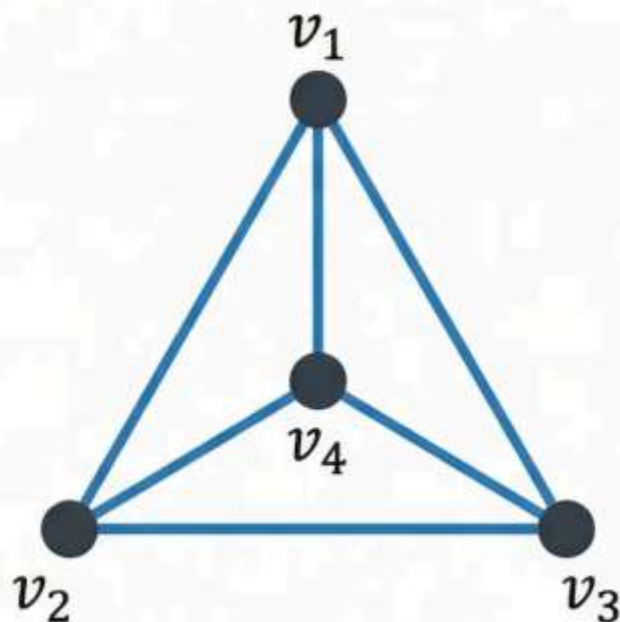
常见完全图 K_3, K_4, K_5 演示



$$m = 3$$

$$3(3-1)/2 = 3$$

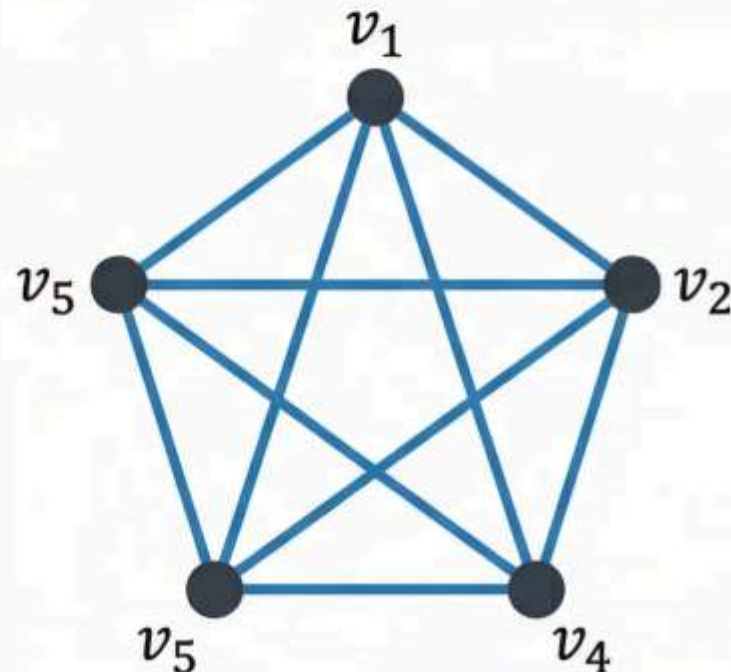
三角形 (Triangle)



$$m = 6$$

$$4(4-1)/2 = 6$$

四面体投影



$$m = 10$$

$$5(5-1)/2 = 10$$

五角完全图

观察：顶点数从 $3 \rightarrow 4 \rightarrow 5$ ，边数从 $3 \rightarrow 6 \rightarrow 10$
边数增长 = $n(n-1)/2$

偶图 / 二部图 (Bipartite Graph)

- 偶图和性义

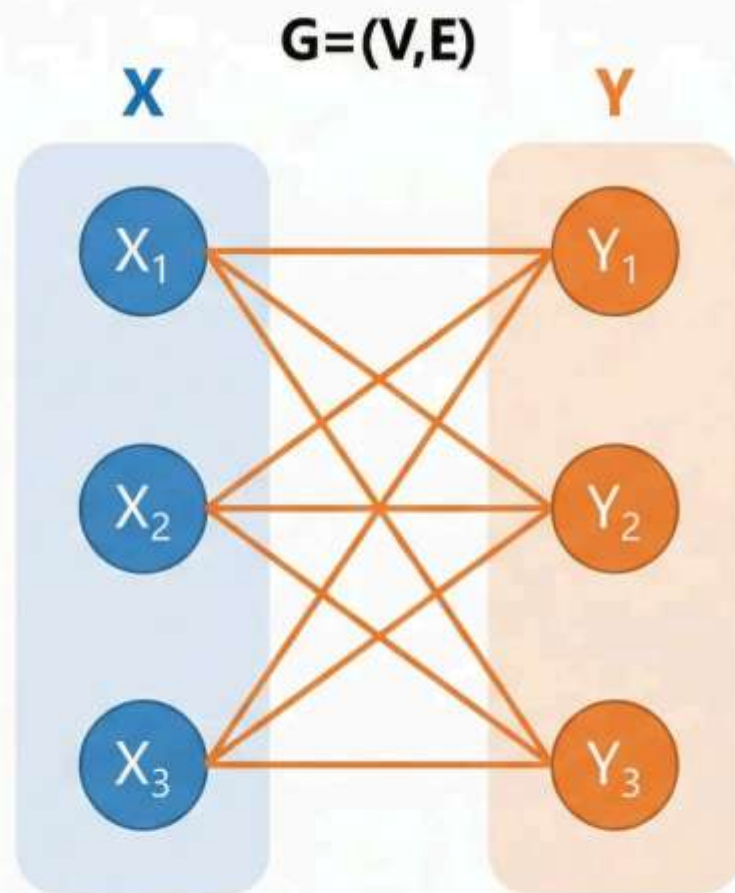
- 一个图 $G=(V,E)$ 的顶点集 V 可以分解为两个非空且不相交的子集 X 和 Y

$$V = X \cup Y \text{ 且 } X \cap Y = \emptyset$$

- 图中的**每条边**都连接 X 中的一个顶点和 Y 中的一个顶点

- 在 X 内部或 Y 内部不存在任何边
- 所有边都是跨集合连接

- **招聘匹配**: 求职者 $\leftrightarrow$ 职位
- **任务分配**: 工作人员 $\leftrightarrow$ 任务
- **推荐系统**: 用户 $\leftrightarrow$ 商品



完全偶图 $K_{m,n}$

如果 X 的每个顶点与 Y 的每个顶点之间都有一条边相连, 则称其为**完全偶图**

- $|V| = m + n$ 个顶点
- $|E| = m \times n$ 条边
- $V = X \cup Y$, 其中 $X \cap Y = \emptyset$

实际应用场景



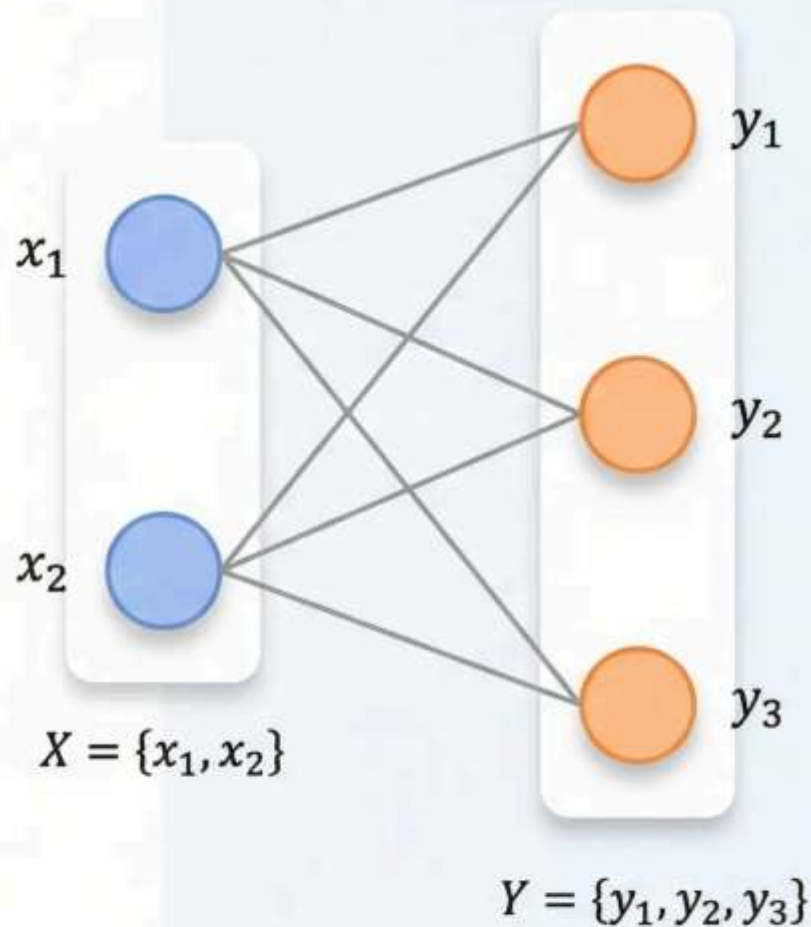
招聘匹配: X 集合是求职者, Y 集合是职位



任务分配: X 集合是工作人员, Y 集合是任务



推荐系统: X 集合是用户, Y 集合是商品



偶图的判定与实例

判定定理

一个图是偶图当且仅当它不包含奇圈（长度为奇数的圈）

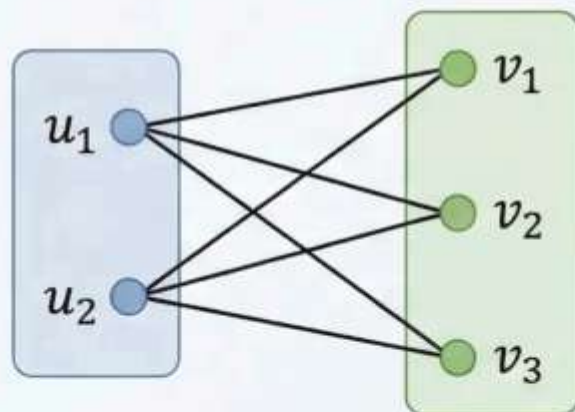
实际应用场景

🤝 **招聘匹配:** 求职者集合 $X \leftrightarrow$ 职位集合 Y

📋 **任务分配:** 工作人员集合 $X \leftrightarrow$ 任务集合 Y

🛒 **推荐系统:** 用户集合 $X \leftrightarrow$ 商品集合 Y

$K_{2,3}$ 标准画法



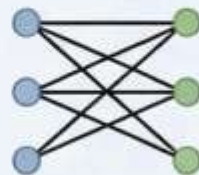
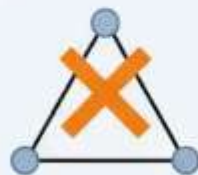
边数计算

$|V| = m + n = 2 + 3 = 5$ 个顶点

$|E| = m \times n = 2 \times 3 = 6$ 条边

快速识别技巧: 寻找图中是否存在三角形 (3-圈)

反例提醒: 完全图 K_n ($n \geq 3$) 都不是偶图



补图 (Complementary Graph)

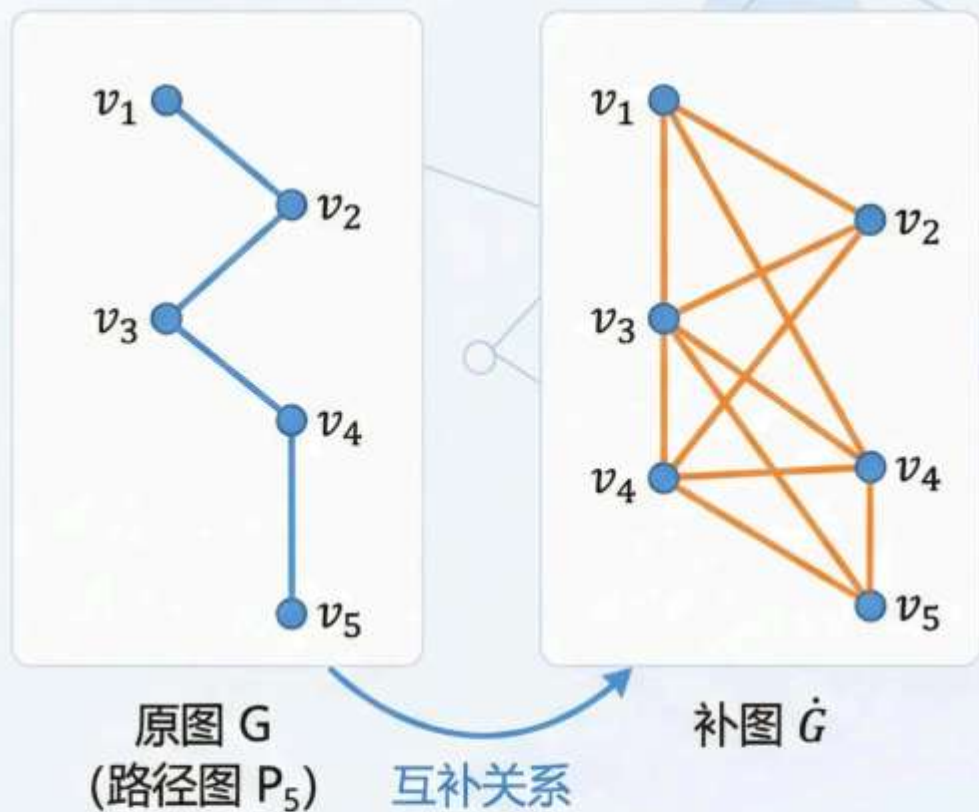
对于简单图 $G=(V,E)$, 其补图 $\dot{G}$ 定义为:

- 在相同顶点集 V 上, 所有不属于 G 的边组成的图

逆向思维: 如果两个顶点在 G 中不相连, 那么它们在 $\dot{G}$ 中相连

公式: $m(G) + m(\dot{G}) = \frac{n(n-1)}{2}$

图的边数与其补图边数之和等于完全图 K_n 的边数



P_5 : 4 条边

$\overline{P_5}$: 6 条边

总计: $4 + 6 = 10 = C(5,2)$

补图的边数守恒性质

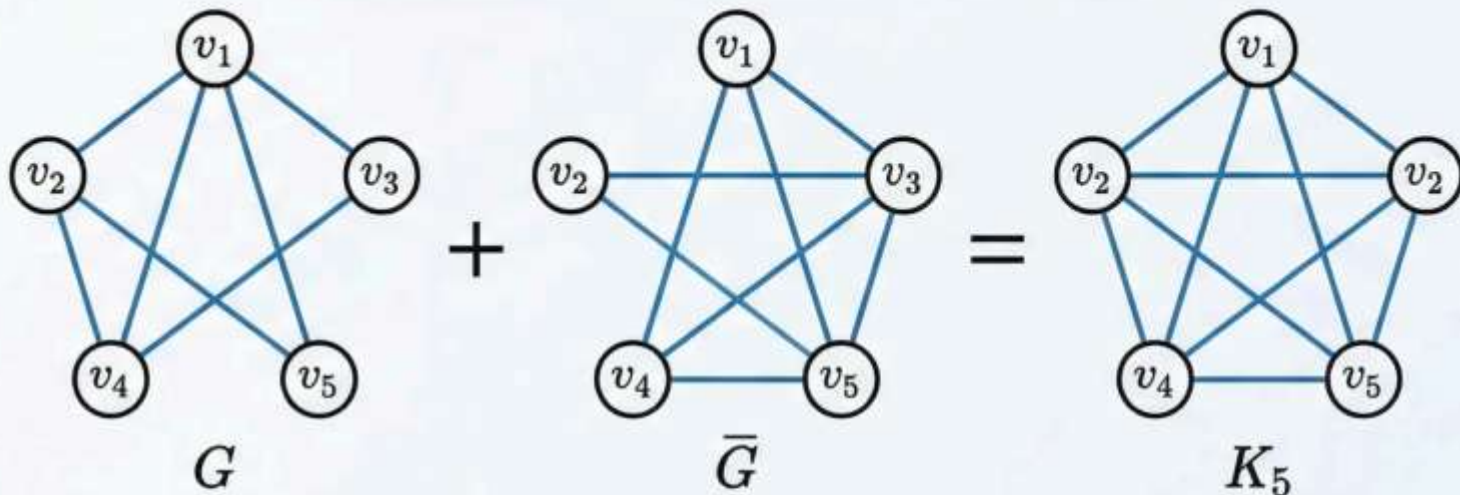
Edge Conservation Property of Complementary Graphs

原图与补图的边数守恒定律

$$m(G) + m(\bar{G}) = \frac{n(n-1)}{2}$$

对于任意 n 阶简单图 G :

- 原图 G 的边数: $m(G)$
- 补图 $\bar{G}$ 的边数: $m(\bar{G})$
- 完全图 K_n 的边数: $\frac{n(n-1)}{2}$



Mathematical Insight

边数守恒的本质

补图包含了原图中所有“缺失”的边

示例：5阶图的边数守恒

若 $m(G) = 4$, 则 $m(\bar{G}) = 10 - 4 = 6$ 验证: $4 + 6 = 10 = \frac{5 \times 4}{2}$

同构的直观理解：结构一致性

图的本质在于连接关系，而非顶点的物理位置

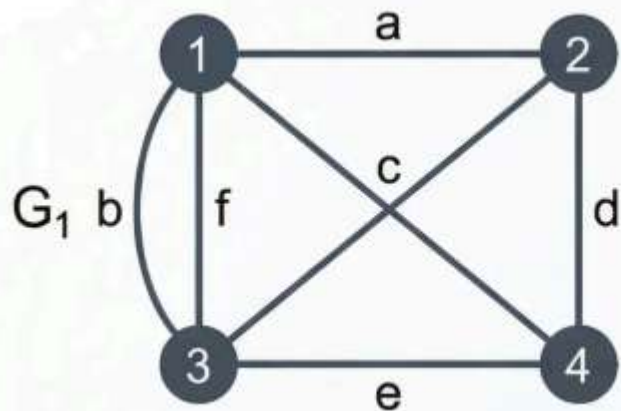


两个图虽然在纸面上看起来绘制得完全不同，顶点的位置或边的弯曲程度各异，但它们的**连接关系**可能完全一致。顶点和边的具体“名称”或“物理位置”并不重要，真正重要的是它们之间的**连接关系**。如果两个图可以“**形变**”为彼此，即通过重排顶点和拉伸/弯曲边而不改变连接方式，那么它们在图论意义上是相同的。

现象分析：看起来不同，实则相同

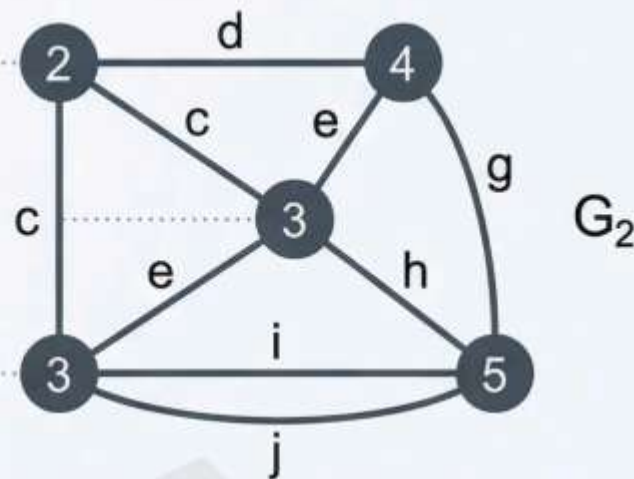
表面差异掩盖结构本质

表现形式 A：规整布局



顶点按几何规律排列

表现形式 B：自由布局



相同顶点，不同位置

结构等价

连接关系完全一致

关键洞察：拓扑结构的不变性

无论顶点如何移动、边如何弯曲，图的本质连接关系保持不变

定义 2：图同构的数学严谨刻画

Definition 2 (Isomorphism):

设 $G_1 = (V_1, E_1)$ 和 $G_2 = (V_2, E_2)$ 是两个图。

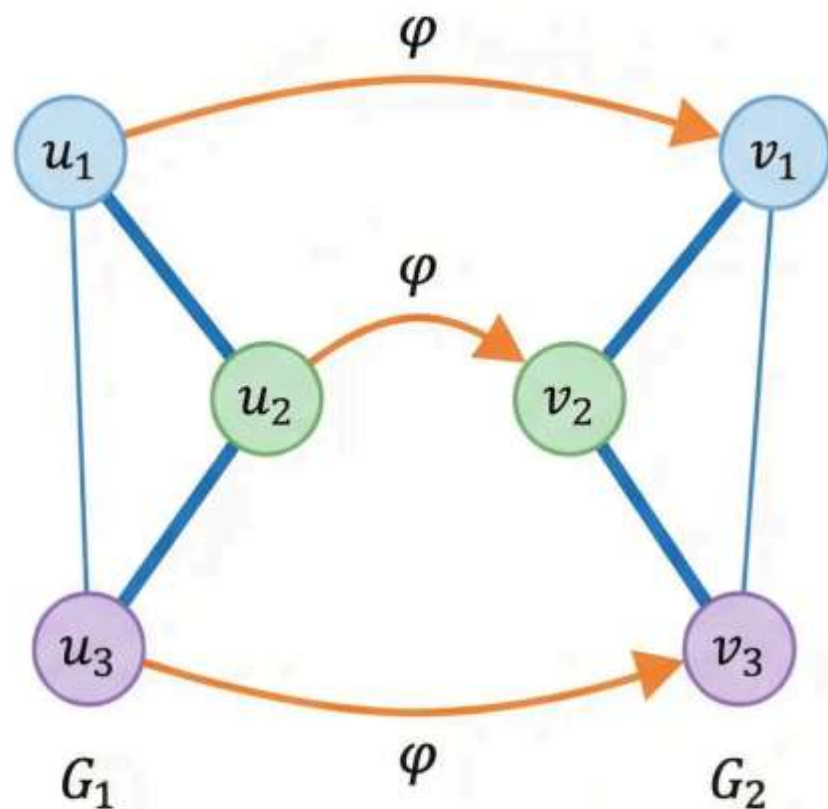
如果存在双射 $\varphi: V_1 \rightarrow V_2$, 使得:

- 对任意 $u, v \in V_1$, 边 $uv \in E_1$ 当且仅当边 $\varphi(u)\varphi(v) \in E_2$, 且边的重数保持相同。

则称 G_1 和 G_2 **同构**, 记为 $G_1 \cong G_2$

Key Requirements Section

- 双射的含义:
- **单射** (Injective) : V_1 中不同顶点映射到 V_2 中不同顶点
- **满射** (Surjective) : V_2 中每个顶点都有 V_1 中的顶点对应
- **邻接关系保持**: 相邻关系在映射下完全保持
- **重数保持**: 对于多重图, 边的重数必须一致



Visual Demonstration of Structure Preservation
($G_1 \cong G_2$)

同构映射的核心要素

核心概念

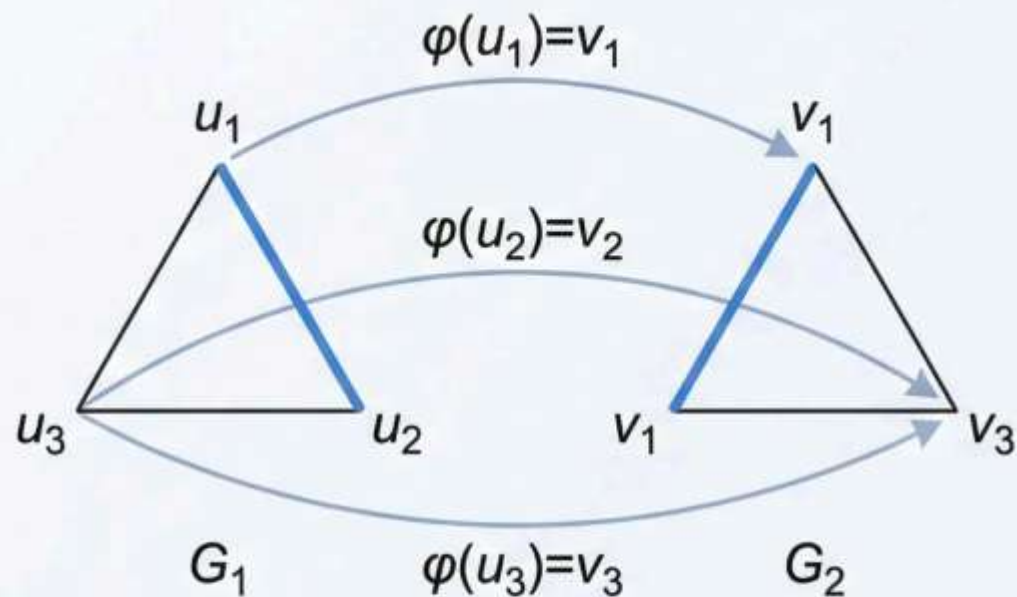
邻接保持的双向性

双射 $\varphi: V_1 \rightarrow V_2$ 必须满足:

$$uv \in E_1 \Leftrightarrow \varphi(u)\varphi(v) \in E_2$$

- $\rightarrow$ 方向: G_1 中相邻 $\rightarrow G_2$ 中对应点也相邻
- $\leftarrow$ 方向: G_2 中相邻 $\rightarrow G_1$ 中原点也相邻

视觉演示



关键洞察: 缺少任一方向的保持性, 图将不同构

判定同构的逻辑步骤

建立双射映射并逐一验证边的对应关系

Methodology

步骤 1: 预备检查

- 验证阶数相等: $|V_1| = |V_2|$
- 验证边数相等: $|E_1| = |E_2|$
- 检查度序列一致性

步骤 2: 建立双射映射

- 构造函数 $\varphi: V_1 \rightarrow V_2$
- 确保映射的单射性 (一对一)
- 确保映射的满射性 (覆盖全部)

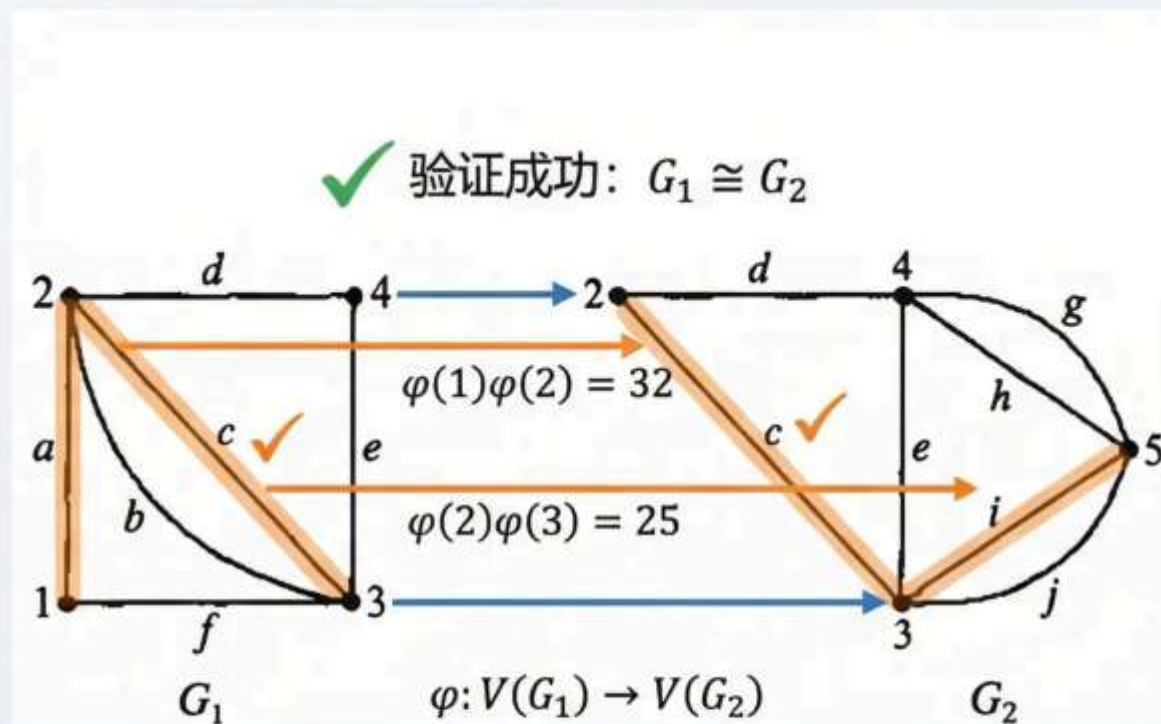
步骤 3: 验证边的保持性

- 对于 G_1 中每条边 $uv \in E_1$
- 检查 $\varphi(u)\varphi(v) \in E_2$
- 逐一验证所有边对应关系

步骤 4: 完整性确认

- 确认映射保持所有结构特性
- 验证重边数量一致 (如适用)
- 得出同构结论

Visual Demonstration



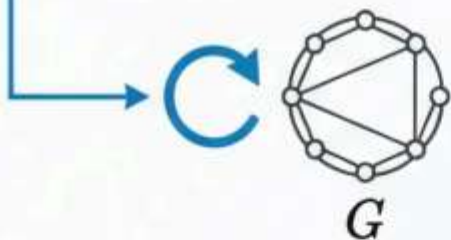
同构关系的等价性质

Equivalence Properties of Graph Isomorphism

图同构 $\cong$ 是图集上的等价关系 (Equivalence Relation)

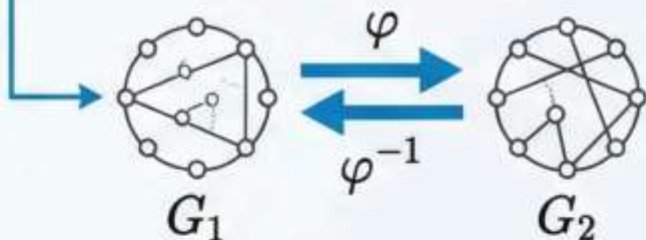
1. 自反性 (Reflexivity)

任何图 G 都与自身同构: $G \cong G$
恒等映射 $\varphi(v) = v$ 保持所有邻接关系



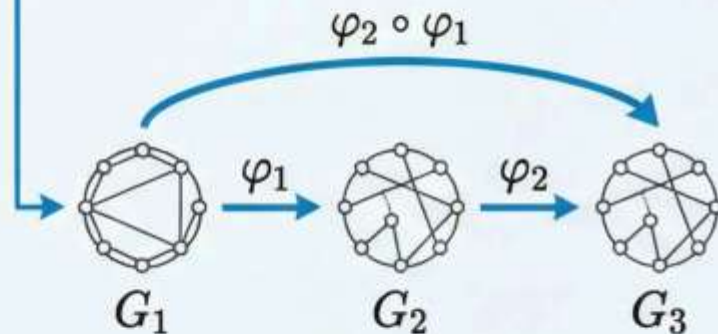
2. 对称性 (Symmetry)

若 $G_1 \cong G_2$, 则 $G_2 \cong G_1$
同构映射 φ 的逆映射 φ^{-1} 也是同构映射



3. 传递性 (Transitivity)

若 $G_1 \cong G_2$ 且 $G_2 \cong G_3$, 则 $G_1 \cong G_3$
同构映射的复合 $\varphi_2 \circ \varphi_1$ 仍是同构映射



Conclusion: 这些性质将所有图划分为互不相交的等价类, 每个等价类中的图都具有相同的抽象结构

标定图 (Labeled Graph)

定义： 图的顶点带有唯一的标识符或编号

关键特征

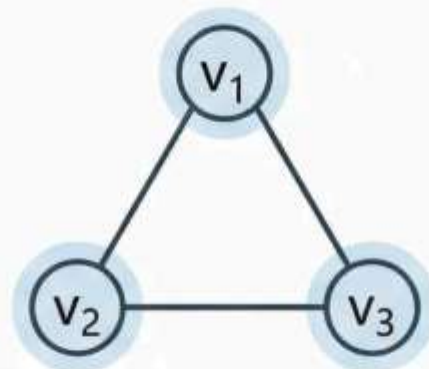
- 每个顶点都有其特定的‘身份’
- 顶点可通过标识符精确区分
- 适用于具体网络实例的表示

应用实例

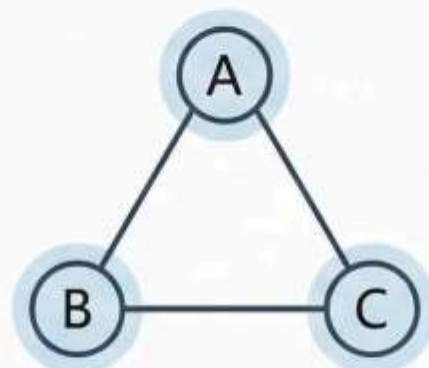
 **计算机网络：** 每个路由器有唯一 **IP地址**

 **社交网络：** 每个用户有唯一 **ID**

 **数据库系统：** 每个节点有**主键标识**



标定三角图：顶点有明确标识



相同结构，不同标定

重要区别： 标定图关注顶点的具体身份

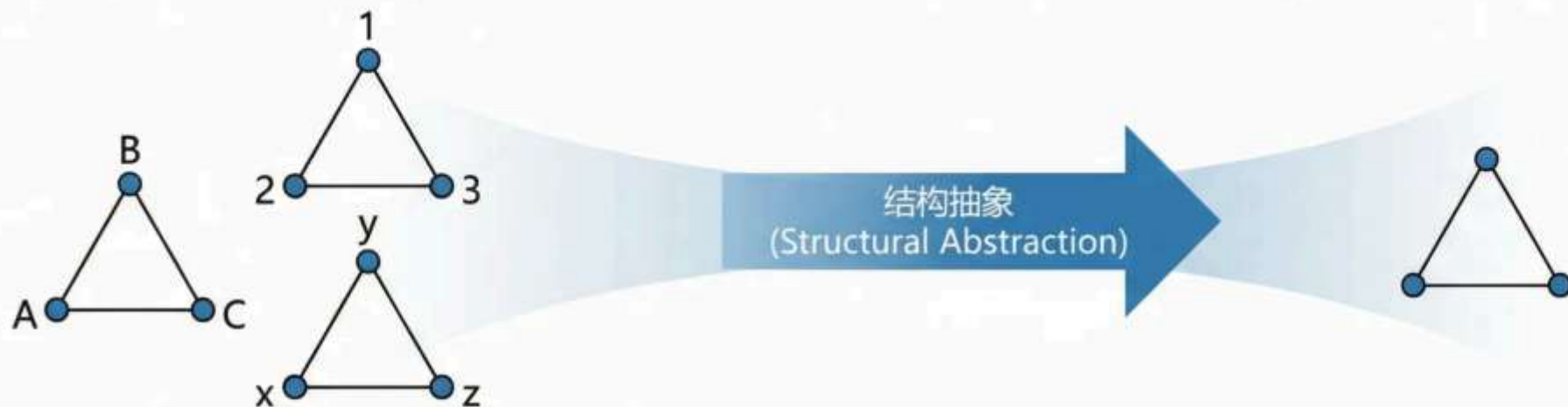
非标定图 (Unlabeled Graph)

非标定图定义

顶点没有唯一标识符的图，我们只关心其结构特性

- **核心特点：**所有同构的图被视为同一种非标定图
- **数学含义：**非标定图是同构关系下的等价类

- **抽象层次：**从具体实例到结构本质的升华
- **等价关系：**将无限多个标定图归类为有限的结构类型
- **研究价值：**专注于图的拓扑性质而非偶然标记



关键理解：非标定图回答“有多少种不同的结构？”而非“有多少个不同的图？”

- **计数问题：**研究 n 阶图的结构种数时，通常指非标定图的数量
- **分类研究：**图论中的很多基本问题都基于结构分类

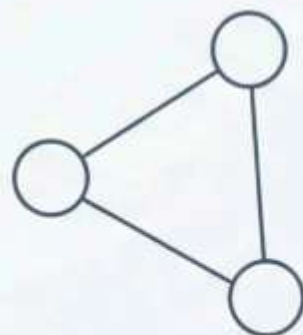
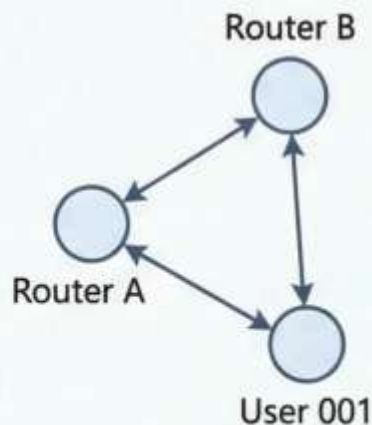
标定与非标定的建模差异

标定图：具体实例建模

- 每个顶点有唯一身份标识
- 适用于具体系统分析
- 网络配置与管理
- 用户关系追踪

应用场景：

- 路由器网络（IP地址标识）
- 社交网络（用户ID系统）
- 数据库关系图



非标定图：结构模式研究

- 关注拓扑结构本质
- 算法复杂度分析基础
- 图的分类与计数
- 理论性质证明

研究价值：

- 网络拓扑类型分析
- 算法设计的通用性
- 数学定理的普适性

为什么非标定图更具研究价值？

- ⚠ **抽象化优势：**剥离偶然细节，聚焦本质结构
- ⚠ **算法通用性：**一种结构对应多个实例，算法具有普适性
- ⚠ **理论深度：**数学性质不依赖具体标签，更具理论意义

同构的必要条件（快速初判）

若 $G_1 \cong G_2$ ，则它们必须满足以下条件：

1. **阶数相同**： $n_1 = n_2$ （顶点的数量必须相同）
2. **边数相同**： $m_1 = m_2$ （边的数量必须相同）
3. **度序列相同**：两个图的度序列（所有顶点的度数按非增序排列）必须完全一致：不仅有相同数量的顶点，而且每个度数的顶点数量也相同。这意味着，如果一个图中有三个度数为 4 的顶点，另一个图也必须恰好有三个度数为 4 的顶点，且所有其他度数的分布也必须完美匹配。

注意

上述条件只是同构的**必要条件**，而不是**充分条件**！

这意味着，即使两个图满足所有这些条件，它们也可能不同构。

视觉示例：满足必要条件，但不同构的图

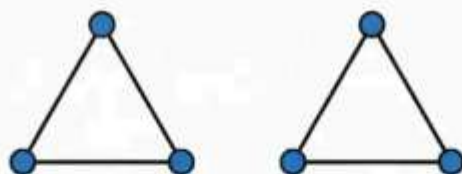


图 G_1

阶数 (n) : $n(G_1) = 6, n(G_2) = 6$

边数 (m) : $m(G_1) = 6, m(G_2) = 6$

度序列: $(2, 2, 2, 2, 2, 2)$ 和 $(2, 2, 2, 2, 2, 2)$

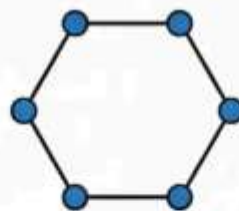


图 G_2

阶数 (n) : $n(G_1) = 6, n(G_2) = 6$

边数 (m) : $m(G_1) = 6, m(G_2) = 6$

度序列: $(2, 2, 2, 2, 2, 2)$ 和 $(2, 2, 2, 2, 2, 2)$

← 相同的必要条件，但不同构 →

关键区别：连通性（左侧不连通，右侧连通）

警示：必要条件非充分条件

即使两个图满足**所有必要条件**，它们也可能**不同构**！

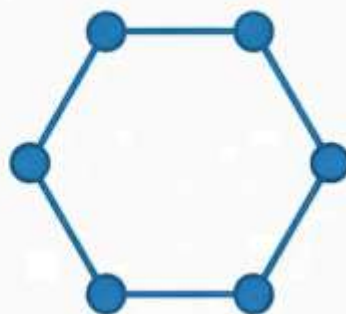
必要条件 Recap:

- 若 $G_1 \cong G_2$ ，则必须满足：
 - 阶数相同： $n_1 = n_2$
 - 边数相同： $m_1 = m_2$
 - 度序列相同

反例展示:



左图：两个独立的 K_3 (三角形)
 $n = 6, m = 6$
度序列：(3, 3, 3, 3, 3, 3)
特点：不连通，包含3-圈



右图：一个 C_6 (六边形)
 $n = 6, m = 6$
度序列：(2, 2, 2, 2, 2, 2)
特点：连通，无3-圈



度序列相同 $\neq$ 图同构

实例研讨：判定不同构的实战

基于教材习题 1-27 的系统分析

判定步骤 (Determination Steps)

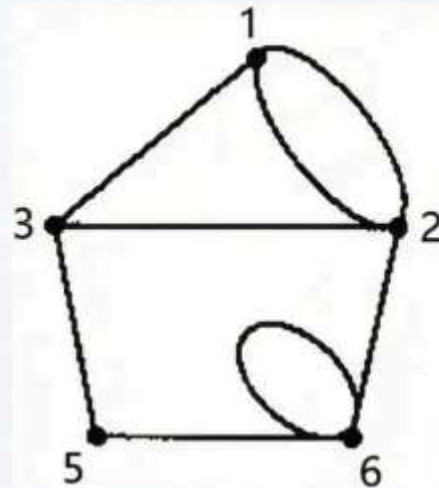
1 ☐ **阶数检查**
计算两图的顶点数量

2 ☐ **边数检查**
统计两图的边数总量

3 ☐ **度序列比较**
分析各顶点度数分布

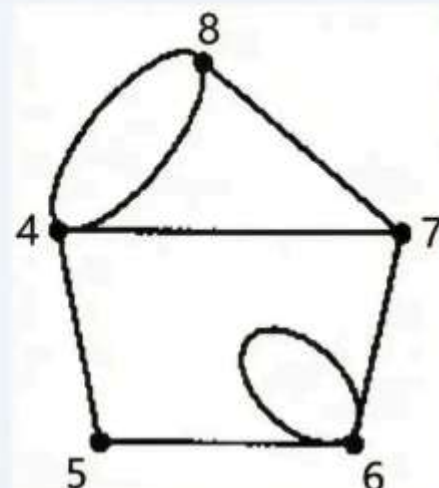
4 ☐ **结构特征识别**
观察环路、连通性等深层特征

顶点数: 8, 边数: 12



度序列: (3,3,3,3,3,3,3,3)

顶点数: 8, 边数: 12



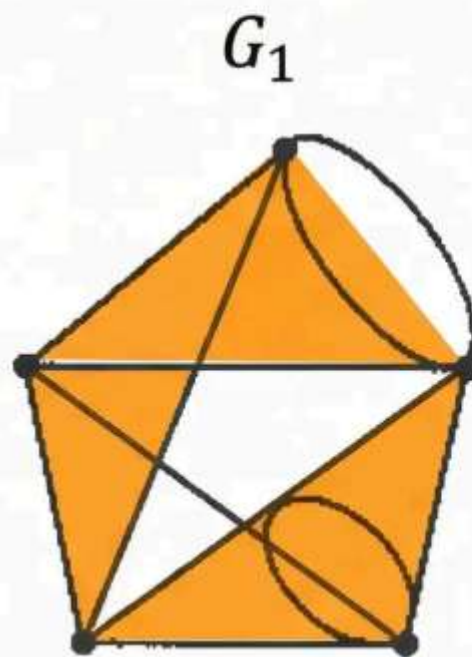
度序列: (3,3,3,3,3,3,3,3)

记住：必要条件不等于充分条件！

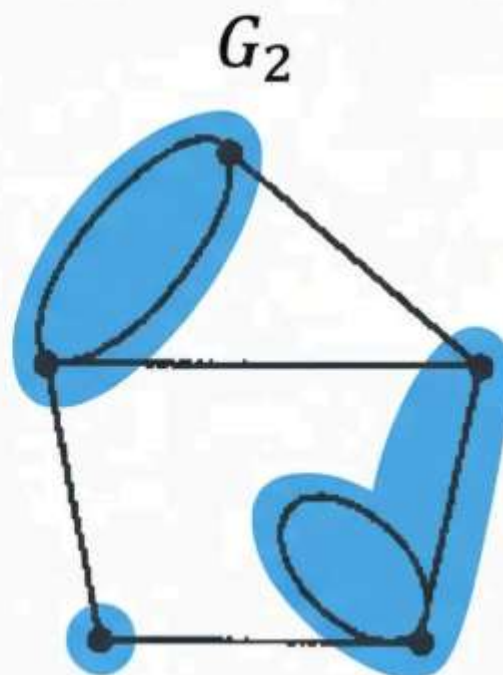
深度分析：利用圈结构破解同构难题

圈结构分析法

- 检查 3-圈（三角形）的数量
- 同构图必须具有相同的圈分布
- 圈结构是拓扑不变量
- 如果图 G_1 包含 k 个三角形
而图 G_2 包含 m 个三角形 ($k \neq m$)
- 则 $G_1 \neq G_2$ (不同构)



包含多个三角形



二部图：无奇圈

结论： 由于两图的三角形数量不同，因此结构本质不同
判定结果： $G_1 \neq G_2$

研讨总结：同构判定的逻辑闭环

Phase 1: 快速预筛选 (Rapid Pre-screening)



Phase 2: 结构特征分析 (Structural Feature Analysis)



Phase 3: 深度验证 (Deep Verification)



Key Insights Box

- **效率原则:** 从简单到复杂, 逐步筛选
- **逻辑严谨:** 必要条件先行, 充分验证后续
- **实战价值:** 大多数情况在前两阶段即可判定

思考提示: 在实际判定中, 约80%的情况可在Phase 1完成判断

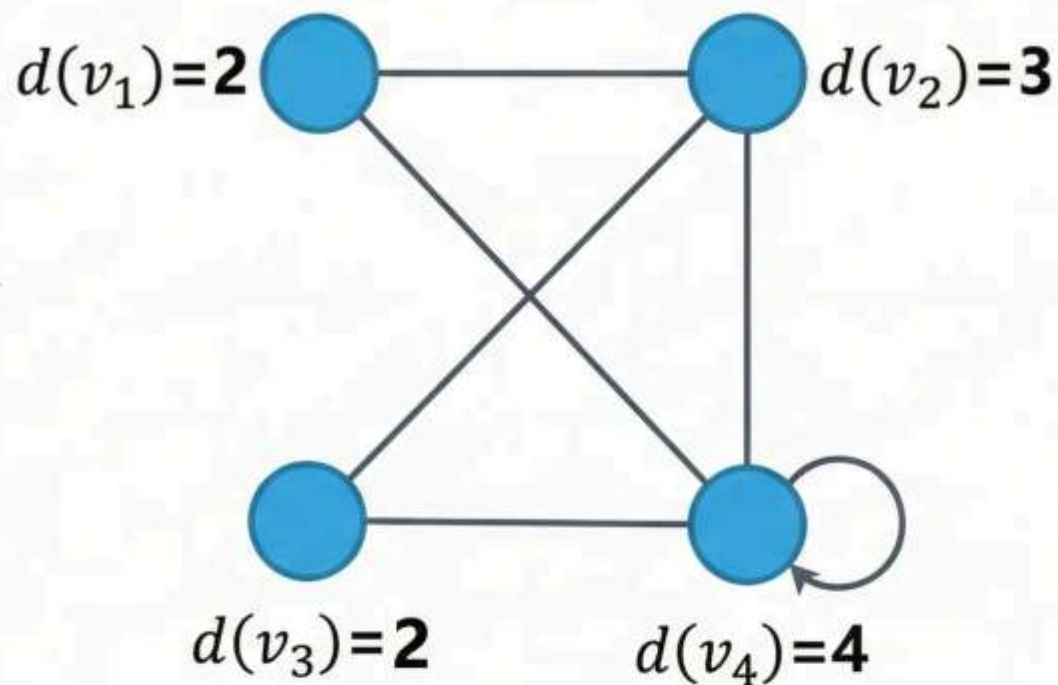
顶点的度 (Degree) 定义

顶点 v 的度 $d(v)$ 指与其关联的边的数目

- 量化顶点的连接重要性
- 反映节点在网络中的中心性

$d(v)$ = degree of vertex v
= degree v

重要提示：环的特殊处理：自环计算两次



度的特殊处理：环的计算

特殊处理：自环对度数的影响

在计算顶点度数时，每个自环必须为该顶点贡献**两次**计数

对于顶点 v 上的自环 $e = (v, v)$ ：

- 该环对 $d(v)$ 的贡献 = **+2**
- 而普通边 $e = (u, v)$ 对每个端点的贡献 = **+1**

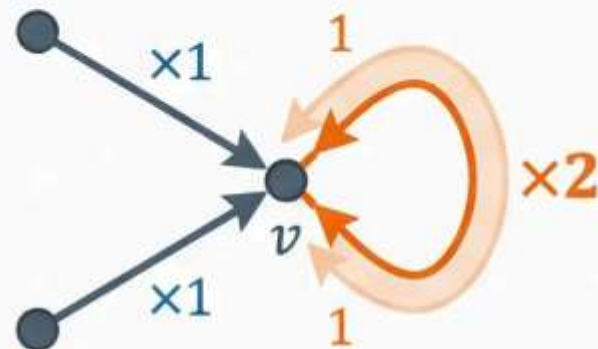
示例：顶点 v 连接 2 条普通边 + 1 个自环

则 $d(v) = 2 \times 1 + 1 \times 2 = 4$

2 (普通边数) $\times$ 1 (每边贡献)

1 (自环数) $\times$ 2 (每环贡献)

Visual Demonstration



! 常见错误：将自环只计算一次

正确理解：自环连接顶点到自身，涉及该顶点两次

度的极值：最大度 Δ 与最小度 δ

最大度 (Maximum Degree)

定义: $\Delta(G) = \max\{d(v) : v \in V(G)\}$

图中所有顶点度数的最大值

含义: 反映图中连接最密集的顶点

最小度 (Minimum Degree)

定义: $\delta(G) = \min\{d(v) : v \in V(G)\}$

图中所有顶点度数的最小值

含义: 反映图中连接最稀疏的顶点

重要性质

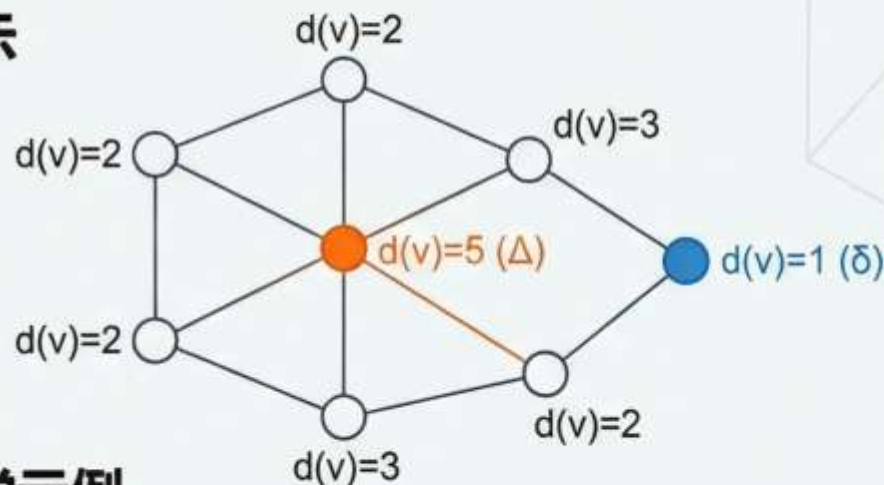
关系: $0 \leq \delta(G) \leq \Delta(G) \leq n - 1$

含义: 度数的分布范围决定了图的连接特性

重要性质

含义: $\Delta(G) = \delta(K_n) \leq n-1$ (极端的度数分布)

图示



数学示例

完全图示例

K_n : $\Delta(K_n) = \delta(K_n) = n - 1$ (所有顶点度数相同)

星图示例

$K_{1,n-1}$: $\Delta = n - 1, \delta = 1$ (极端的度数分布)

关键洞察

- 连接强度分析: $\Delta - \delta$ 的值反映图的连接均匀性
- 网络特征: 最大度高的图通常有"中心节点"特征
- 应用价值: 在网络分析中用于识别关键节点和脆弱环节

顶点的分类：奇点与偶点

Definitions

奇点 (Odd Vertex)

度数为奇数的顶点

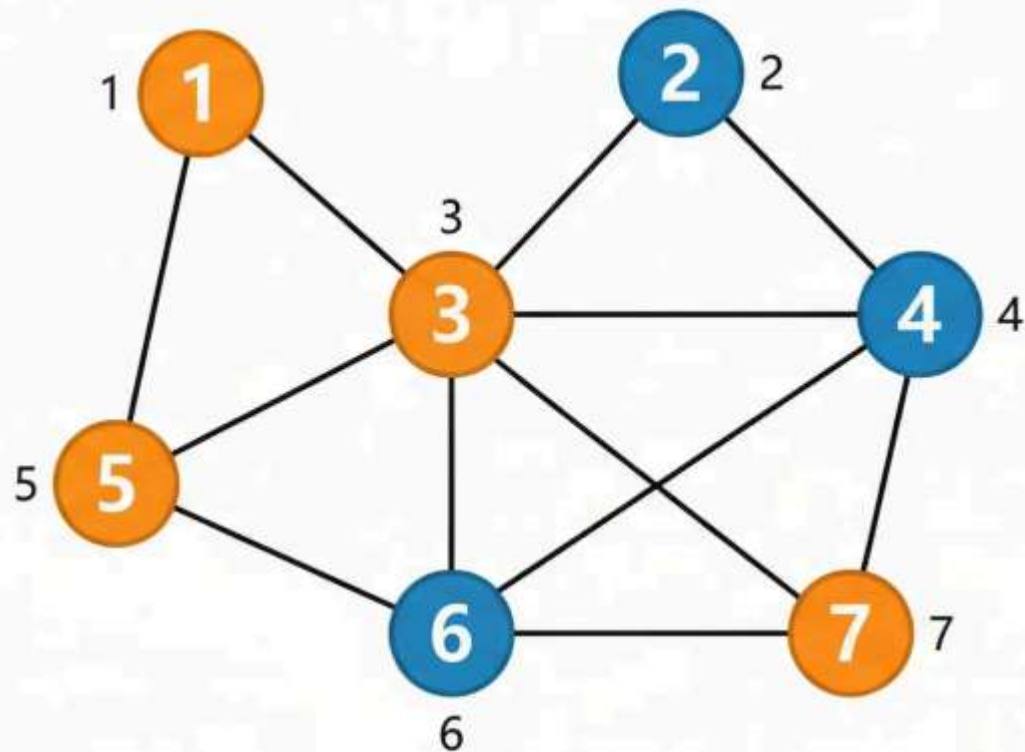
度数为 1, 3, 5, 7, ... 的顶点

偶点 (Even Vertex)

度数为偶数的顶点

度数为 0, 2, 4, 6, ... 的顶点

奇偶性分类是研究欧拉路径、哈密顿回路等经典问题的基础



● 奇点 (Odd Degree)
● 偶点 (Even Degree)

Hint: Euler paths relate to odd vertices

特殊图： k -正则图 (k -regular)

定义： 图中所有顶点的度数都相同且等于 k 的简单图

$$d(v) = k \text{ for all } v \in V(G)$$

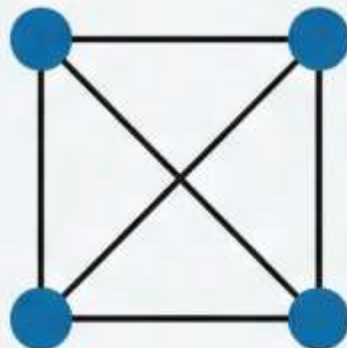
关键性质

- 完美的结构对称性
- 每个顶点的连接模式完全一致
- 负载均衡的理想网络拓扑
- 高度规整的数学结构

数学洞见： 对于 k -正则图： $\sum d(v) = nk = 2m$

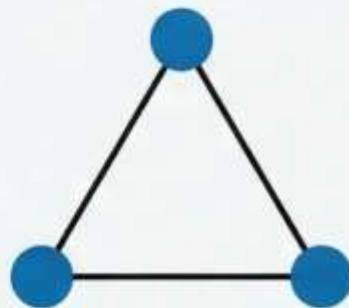
$$\text{因此边数 } m = \frac{nk}{2}$$

K_4 是 3-正则图



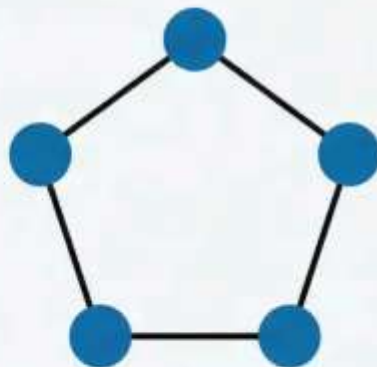
每个顶点度数 = 3

K_3 是 2-正则图



每个顶点度数 = 2

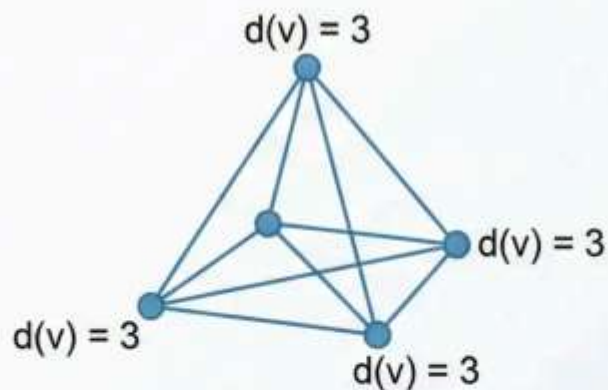
C_5 是 2-正则图



环图的规律性

正则图实例展示

K_4 作为 3-正则图



4 个顶点, 每个顶点度数 = 3

总边数: 6 条边

边数公式验证: $\frac{4 \times 3}{2} = 6$

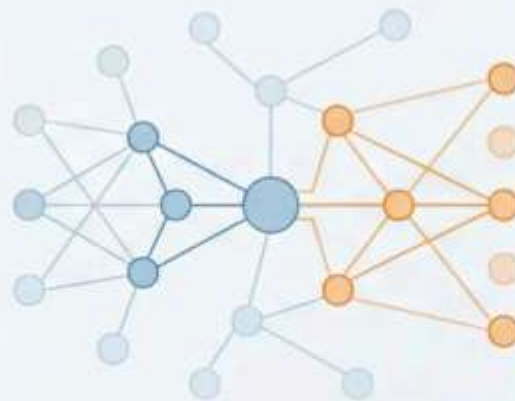
每个顶点都与其他 3 个顶点相连

完美的对称性与规整结构

K_4 是 3-正则图

$$\sum d(v) = 4 \times 3 = 12 = 2 \times 6$$

网络负载均衡的理想模型



- **均匀分布:** 每个节点处理相同的连接负载
- **高可靠性:** 多条路径确保网络冗余
- **对称设计:** 无单点故障的网络拓扑
- **扩展性:** 规则结构便于网络扩展

定理 2：图论第一定理（握手定理）

Theorem 2 (The Handshaking Lemma): 图 $G=(V, E)$ 中所有顶点的度之和等于边数 m 的 2 倍：
$$\sum_{v \in V} d(v) = 2m$$

直观描述：这个定理被称为'握手定理'，因为它可以用一个形象的比喻来理解：在一个聚会上，如果每个人都与某些人握手，那么所有握手次数的总和一定是偶数，因为每次握手都涉及两个人，贡献了两次'握手'。

重要性：握手定理是图论中最基本、最核心的定理之一，它揭示了图的顶点度和边数之间固有的数学关系。它不仅是许多图论证明的基石，也是判断一个给定的图是否存在的必要条件。

握手定理的证明：边的双重贡献

核心证明逻辑

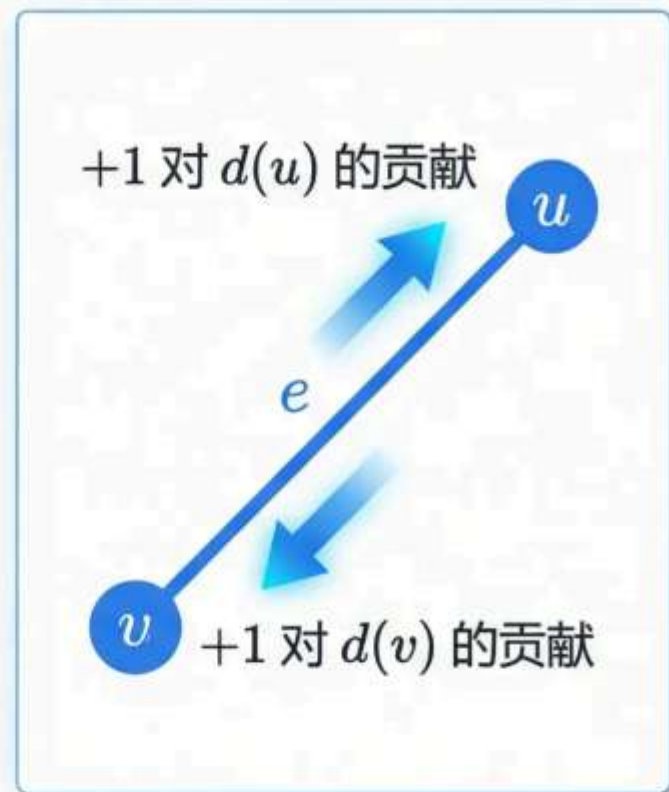
- Step 1: 考虑图 $G = (V, E)$ 中的每一条边 $e \in E$
- Step 2: 每条边都恰好连接两个顶点
- Step 3: 在计算所有顶点度数之和 $\sum_{v \in V} d(v)$ 时

每条边 $e = uv$ 贡献：
 $d(u)$ 增加 1, $d(v)$ 增加 1

总贡献：每条边贡献 2

结论： $\sum_{v \in V} d(v) = 2m$

每条边被计算恰好两次



证明细节：累加过程的逻辑

考虑图 $G=(V,E)$ 中的每条边及其贡献

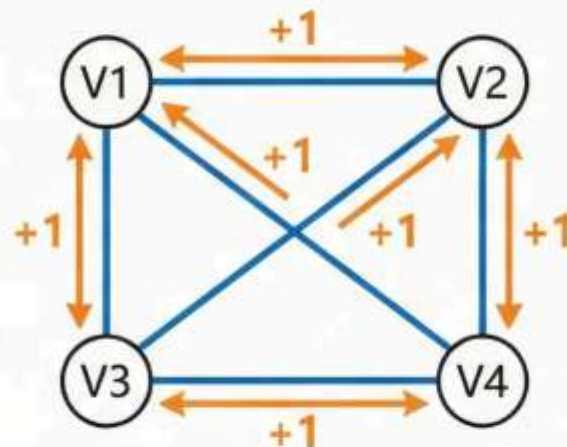
每条边 $e=uv$ 对总度数的双重计数

累加过程： $\sum d(v) =$ 每条边的贡献之和

$$\sum_{v \in V} d(v) = \sum_{e \in E} (\text{贡献值})$$

对于边 $e=uv$:

- 在计算 $d(u)$ 时贡献 +1
- 在计算 $d(v)$ 时贡献 +1
- 总贡献 = 2



Accumulation Table

边 (e)	对 $d(u)$ 贡献	对 $d(v)$ 贡献	总贡献
e1 (V1,V2)	+1 (V1)	+1 (V2)	2
e2 (V2,V3)	+1 (V2)	+1 (V3)	2
e3 (V3,V4)	+1 (V3)	+1 (V4)	2
e4 (V4,V1)	+1 (V4)	+1 (V1)	2
总和 (Σ)	$\Sigma d(u)$ 部分	$\Sigma d(v)$ 部分	$2 * E $

总结：每条边 e 在累加过程中被计算了两次，分别贡献给其两个端点的度数，因此总度数之和为边数的两倍。

握手定理对环的普适性

引导问题：如果图中包含环，握手定理 $\sum_{v \in V} d(v) = 2m$ 还成立吗？

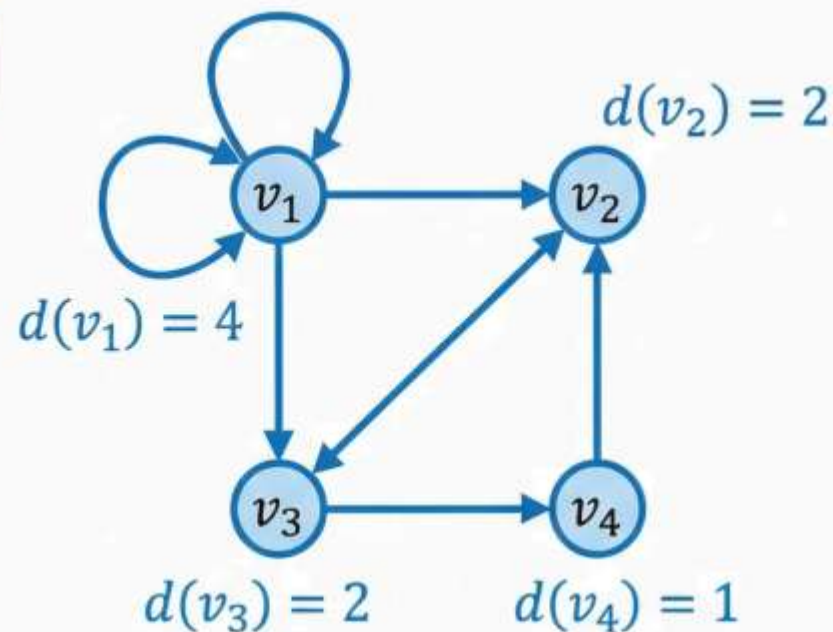
明确答案：完全成立！环的处理方式保证了定理的一致性。

核心原理

- 环连接顶点 v 自己，对 $d(v)$ 贡献度数 2
- 这等价于一条普通边连接两个不同顶点时各贡献度数 1
- 总贡献依然是每条边（包括环）贡献 2 到总度数

数学验证

- 设图 G 有 m 条边（包括环）
- 每条边（无论是否为环）都恰好贡献 2 到 $\sum d(v)$
- 因此 $\sum_{v \in V} d(v) = 2m$ 恒成立



重要提醒：环在度数定义中被计算两次的规定，正是为了保持握手定理的普适性！

推论 1：奇点个数的奇偶性

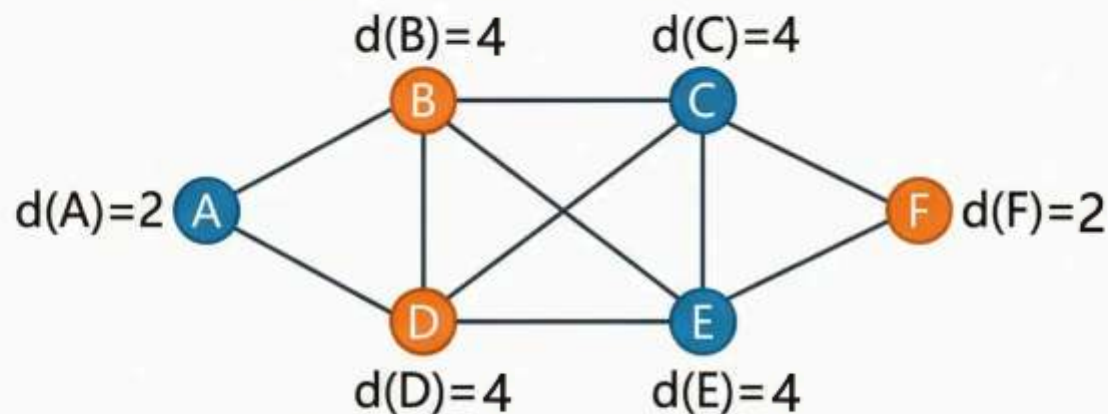
Corollary 1：在任何图中，奇点的个数必为偶数

握手定理的直接推论

每条边连接两个顶点，总度数必为偶数 ($\sum d(v) = 2m$)

奇数个奇点会导致总度数为奇数，产生矛盾

逻辑链： $2m$ 是偶数 $\rightarrow$ 奇点度数和必为偶数 $\rightarrow$ 奇点个数必为偶数



证明思路：

Step 1: 将顶点分为奇点集 V_1 和偶点集 V_2

Step 2: 应用握手定理的代数性质

Step 3: 利用奇偶数运算规律

详细证明见下一页

为什么这很重要？

- 图存在性的快速判定工具
- 构造图时的基本约束条件
- 许多图论证明的基石

推论 1 证明：顶点集的奇偶划分

6

将图 G 的顶点集 V 分成两个不相交的子集：

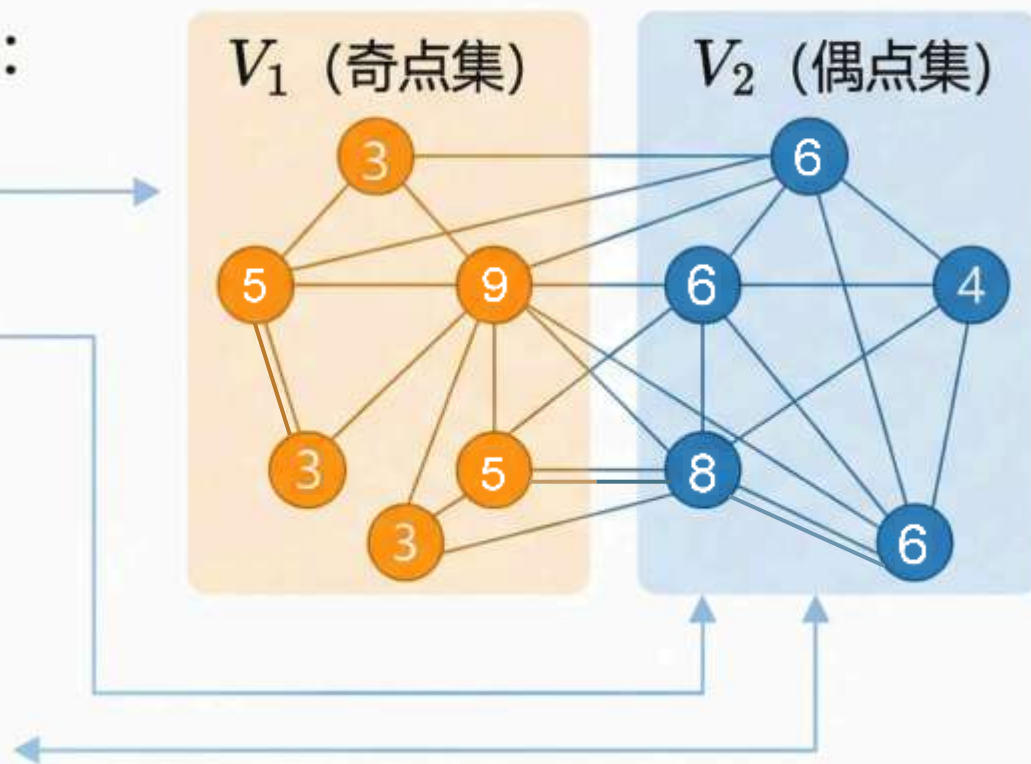
V_1 (奇点集)：所有度数为奇数的顶点

V_2 (偶点集)：所有度数为偶数的顶点

Step 2: 握手定理的应用

$$\begin{aligned} \sum_{v \in V} d(v) &= 2m \\ &= \sum_{v \in V_1} d(v) + \sum_{v \in V_2} d(v) = 2m \end{aligned}$$

由于 $2m$ 是偶数，我们可以分析各部分的奇偶性



推论 1 证明：代数恒等式推导

通过偶数减去偶数必为偶数的逻辑，推导奇点度数之和的性质

证明步骤 3：代数恒等式分析

$$\sum_{v \in V_1} d(v) + \sum_{v \in V_2} d(v) = 2m$$

已知条件分析：

- 左侧：奇点度数之和 $\sum_{v \in V_1} d(v)$ (待确定奇偶性)
- 右侧：偶点度数之和 $\sum_{v \in V_2} d(v)$ (必为偶数)
- 总和： $2m$ (握手定理保证为偶数)

关键推理：

偶数 - 偶数 = 偶数



$$\sum_{v \in V_1} d(v) = 2m - \sum_{v \in V_2} d(v)$$



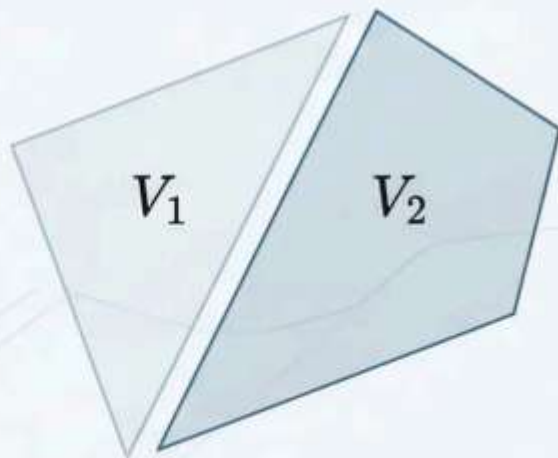
因此 $\sum_{v \in V_1} d(v)$ 必为偶数

代数核心洞察

为什么偶点度数之和必为偶数？

偶数 + 偶数 + ... + 偶数 = 偶数

这是基本的数论性质！



推论 1 证明：最终逻辑闭环

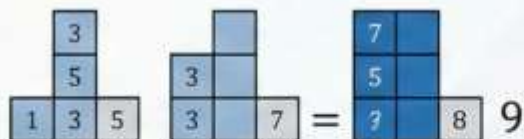
Corollary 1 Proof: Final Logical Closure

奇数的算术性质

- ✓ 奇数个奇数相加 → 结果为奇数
- ✓ 偶数个奇数相加 → 结果为偶数

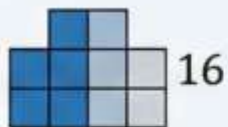
Step 5: 现在考虑奇点集 V_1 中的度数性质

- 每个顶点的度数都是奇数
- 关键观察：奇数的个数决定了总和的奇偶性



例：1 + 3 + 5 = 9
(3个奇数 → 奇数结果)

例：1 + 3 + 5 + 7 = 16
(4个奇数 → 偶数结果)



奇点度数之和 = 偶数 (Established Fact)

假设 $|V_1|$ 为奇数

假设 $|V_1|$ 为偶数

奇数个奇数之和 = 奇数

偶数个奇数之和 = 偶数

矛盾!



反证推理

- 已知： $\sum_{v \in V_1} d(v)$ 必须为偶数
- 假设： $|V_1|$ 为奇数
- 推出：奇数个奇数之和 = 奇数
- 矛盾！因此 $|V_1|$ 必须为偶数



结论：任何图中奇点的个数必为偶数 □

判定应用：图的存在性验证

“奇点个数必为偶数” - 图的快速存在性判定

Step 1

统计度序列中的奇点个数

度序列: (3,3,2,1,1)

分析: 奇点: 3, 3, 1, 1 (共4个奇点)

奇点个数: 4 (偶数)

结论:  通过此项检验

度序列: (5,3,3,2,1)

分析: 奇点: 5, 3, 3, 1 (共4个奇点)

奇点个数: 4 (偶数)

结论:  通过此项检验

Step 2

判断奇点个数的奇偶性

度序列: (4,3,3,3,2,1)

分析: 奇点: 3, 3, 3, 1 (共4个奇点)

奇点个数: 4 (偶数)

结论:  通过此项检验

度序列: (4,3,3,1)

分析: 奇点: 3, 3, 1 (共3个奇点)

奇点个数: 3 (奇数) 

结论:  立即判定为不可图

Step 3

奇数个奇点 → 该图不存在

这是最快速的初步筛选方法 - 无需复杂计算即可排除不可能的度序列

快速判定: (2, 2, 1, 1, 1) 是否可能构成图? _____

实例分析：不存在的图

问题陈述

是否存在一个简单图，所有顶点的度数分别为 $(3, 1, 1, 1)$ ？

分析过程

- 1. 顶点数: $n = 4$
- 2. 度数序列: $(3, 1, 1, 1)$
- 3. 奇点识别: 度数为 3, 1, 1, 1 的顶点都是奇点
- 4. 奇点计数: 共有 4 个奇点



⚠ 违反握手定理推论1

关键点: 奇点个数 = 4 (偶数)

更正: 等等...让我重新检查

实际分析: 奇点个数 = 4 个奇点

状态: 4 是偶数, 满足推论1

状态: 4 是偶数, 满足推论1

附加分析

- 度数之和 = $3 + 1 + 1 + 1 = 6$ (偶数) ✓
- 满足握手定理: $\sum d(v) = 2m$
- 最大度 $\Delta = 3 \leq n - 1 = 3$ ✓

修正示例

新问题: 考虑度数序列 $(3, 1, 1)$ - 3个顶点 分析: 奇点个数 = 3 (奇数) ✗ 结论: 此序列违反推论1, 不可图

推论 2: 正则图的约束条件

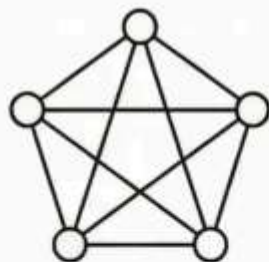
Corollary 2: 正则图的阶数 n 和度数 k 不同时为奇数

对于 k -正则图: $\sum d(v) = nk = 2m$

由于 $2m$ 必须是偶数, 所以 nk 必须是偶数

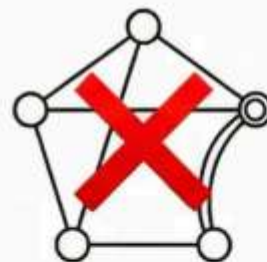
因此 n 和 k 不能同时为奇数

✓ **Valid Example: K_5 ($n=5$ 奇数, $k=4$ 偶数) ✓**



符合约束条件

⊘ **Invalid Example: 假设的5阶3-正则图 ✗**



$n=5$ (奇), $k=3$ (奇)
违反约束

关键洞察: $nk = 2m$ 恒等式的奇偶性分析

当 n 和 k 同时为奇数时, nk 为奇数, 但 $2m$ 必为偶数, 产生矛盾

推论 2 证明：基于 $nk = 2m$ 的分析

Step 1 - 基础等式建立:

设 G 是一个 k -正则图，根据握手定理：

$$\sum_{v \in V} d(v) = 2m$$

由于所有顶点度数均为 k ，因此：

$$nk = 2m$$

Step 2 - 奇偶性分析:

等式 $nk = 2m$ 表明，乘积 nk 必须是偶数
因为右侧 $2m$ 显然为偶数

Step 3 - 矛盾推理:

对于两个整数的乘积为偶数，当且仅当：

- n 是偶数，或者
- k 是偶数，或者
- 两者都是偶数

唯一不可能的情况： n 和 k 同时为奇数

$$\sum_{v \in V} d(v) = 2m \rightarrow nk = 2m$$

必须是偶数



偶数 (Even)

n 是偶数

k 是偶数

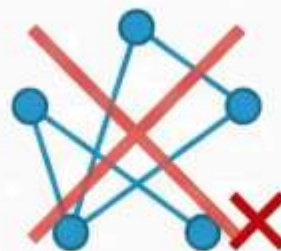
n, k 都是偶数

~~n 是奇数,
 k 是奇数~~

~~$nk = 2m$
(矛盾)~~



Example: $n=5, k=4$
(可行)



Example: $n=5, k=3$
(不可能)

关键洞察：
奇数 $\times$ 奇数 = 奇数，
与 $nk = 2m$ (偶数)
矛盾！

正则图的奇偶匹配逻辑

n 与 k 的奇偶组合分析

组合 1	n 偶数, k 偶数 $\rightarrow nk$ 偶数	✓ 合法
组合 2	n 偶数, k 奇数 $\rightarrow nk$ 偶数	✓ 合法
组合 3	n 奇数, k 偶数 $\rightarrow nk$ 偶数	✓ 合法
组合 4	n 奇数, k 奇数 $\rightarrow nk$ 奇数	✗ 禁止

理论依据

基于握手定理: $\sum d(v) = nk = 2m$

nk 必须为偶数 (因为 $2m$ 总是偶数)

奇数 $\times$ 奇数 = 奇数, 违反此条件

强调重点

在 k -正则图中, 阶数 n 与度数 k 不能同时为奇数

奇 $\times$ 奇 $\rightarrow$ 奇数 $\rightarrow$ 违反 $nk = 2m$

实例验证：正则图的合法性

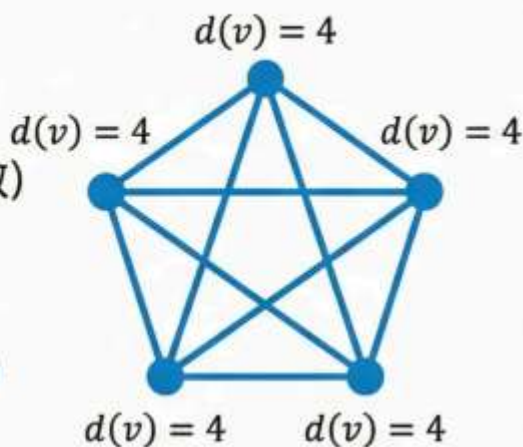
验证具体实例，理解奇偶约束的实际意义

✓ 合法实例：完全图 K_5

- 阶数 $n = 5$ (奇数)
- 度数 $k = 4$ (偶数)
- 结论：符合推论2 (不同时为奇数)

边数公数验证

- 边数公式： $m = \frac{nk}{2} = \frac{5 \times 4}{2} = 10$
- 完全图边数： $C(5,2) = 10$ ✓
- 验证：数学一致性成立



✗ 不可能实例：5阶3-正则图

- 假设阶数 $n = 5$ (奇数)
- 假设度数 $k = 3$ (奇数)
- 违反推论2：同时为奇数！

矛盾说解

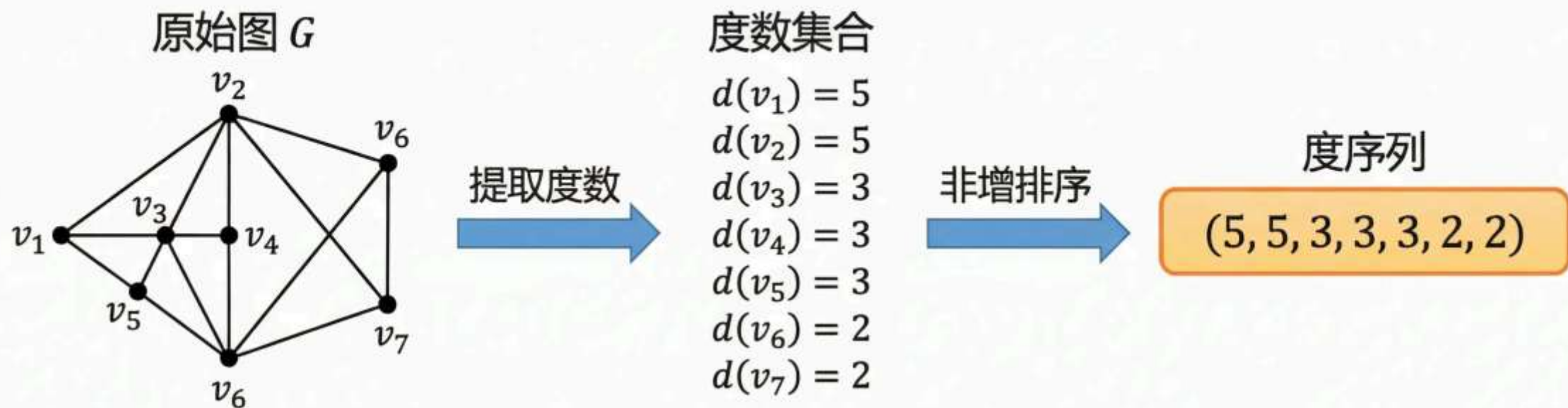
- 总度数： $nk = 5 \times 3 = 15$ (奇数)
- 握手定理要求： $\sum d(v) = 2m$ (必须为偶数)
- 矛盾：奇数 $\neq$ 偶数



推论2的实际意义：正则图的阶数与度数至少有一个必须是偶数，这保证了握手定理的成立。

度序列 (Degree Sequence) 定义

图 G 的各个顶点的度构成的非负整数组 $(d_1, d_2, \dots, d_n)$
通常按非增顺序排列: $d_1 \geq d_2 \geq \dots \geq d_n$



- 度序列是图结构的“指纹”特征
- 同构图必有相同度序列
- 相同度序列不保证图同构

度序列的惯例与意义

排列惯例 (Ordering Convention)

度序列按 **非增顺序** 排列

$$d_1 \geq d_2 \geq \dots \geq d_n$$

便于比较和分析图的结构特征

“拓扑指纹”的概念

度序列是图的“拓扑指纹”

- 反映图的基本连接模式
- 同构图必有相同度序列
- 不同结构可能产生相同指纹 (非唯一性)

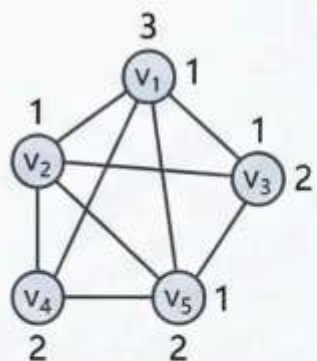
⚠ 注意：相同度序列 ≠ 图同构

实际意义

Analysis Tool: 快速识别图的结构类型

Comparison Basis: 判定图同构的必要条件

Pattern Recognition: 发现图的规律性特征



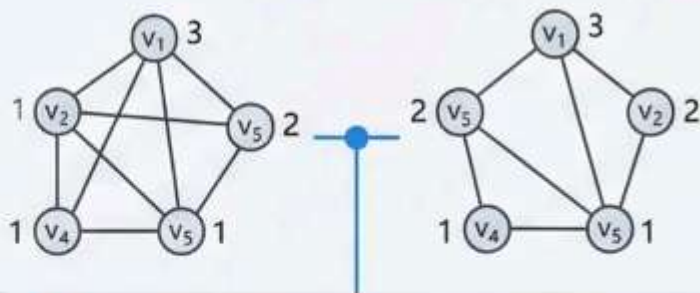
(3, 1, 2, 2, 1)

(3, 2, 2, 1, 1)



拓扑指纹

相同度序列: (3, 2, 2, 1, 1)



⚠ 不同结构但具有相同的“拓扑指纹”

核心问题：可图化 (Graphic) 判定

Problem Definition

核心问题： 给定一个非负整数序列，是否存在一个**简单图**以它为度序列？

Terminology Box

如果存在，则称该序列是**可图的 (Graphic)**

Fundamental Conditions

必要条件：

1. 度数之和为偶数： $\sum d_i$ 必须是偶数
2. 最大度限制： $\Delta \leq n-1$

Visual Examples

Example 1 - Invalid Sequence

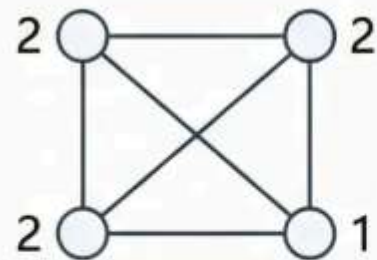
(4, 1, 1, 1)

$n=4, \sum d_i=7$ (奇数)



Example 2 - Valid Sequence

(2, 2, 1, 1)



可图化的必要条件回顾

给定一个非负整数序列 $\pi = (d_1, d_2, \dots, d_n)$, 要判断其是否可图, 必须首先验证以下基本条件:

条件 1: 度数和的偶数性

$$\sum_{i=1}^n d_i \text{ 必须为偶数}$$

理论依据: 握手定理的直接推论

解释: 所有顶点度数之和等于边数的两倍, 故必为偶数

条件 2: 最大度的上界限制

$$\max\{d_1, d_2, \dots, d_n\} \leq n - 1$$

理论依据: 简单图中顶点最多连接其他所有顶点

解释: 在 n 个顶点的简单图中, 任一顶点最多有 $n-1$ 条边

⚠ 重要提醒: 这些条件**仅为必要条件, 非充分条件!** 满足这两个条件的序列仍可能不可图。

违反条件1

(4, 1, 1, 1) ❌

序列 (4, 1, 1, 1) 标注为“违反条件1”, 用**红叉**标记



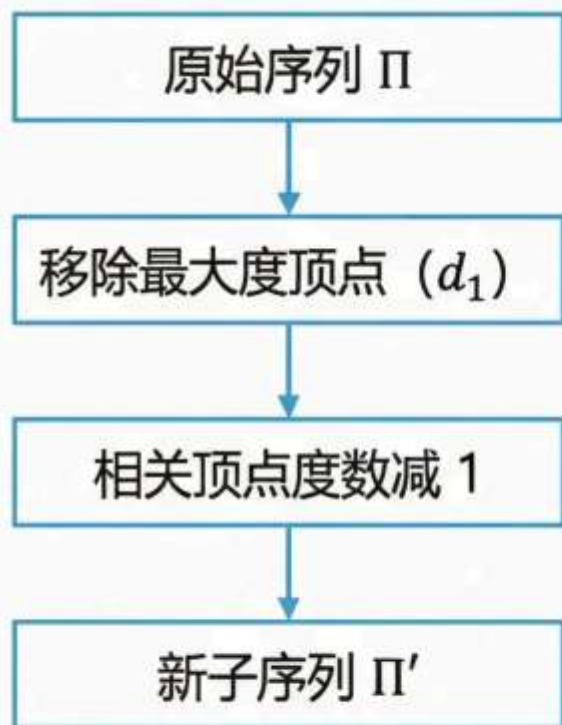
满足必要条件

(2, 2, 1, 1) ✅

序列 (2, 2, 1, 1) 标注为“满足必要条件”, 用**绿勾**标记

定理 3: Havel-Hakimi 算法

Theorem 3 (Havel-Hakimi Theorem): 一个非负整数组 $\Pi = (d_1, d_2, \dots, d_n)$, 其中 $n - 1 \geq d_1 \geq d_2 \geq \dots \geq d_n$, 是可图的**当且仅当**序列 $\Pi' = (d_2 - 1, d_3 - 1, \dots, d_{d_1+1} - 1, d_{d_1+2}, d_{d_1+2}, \dots, d_n)$ 是可图的。



递归缩减思想 (Recursive Reduction Concept)

- 假设度数最大的顶点连接了其他度数最大的 d_1 个顶点
- 从这 d_1 个顶点的度数中各减去 1
- 移除原最大度顶点, 得到新的子序列
- 如果子序列可图, 则原序列可图

算法核心: 将图的构造问题转化为**更小规模的同类问题**

HH 算法步骤：排序与移除

Step 1 详解：算法的起始操作

核心操作流程

- 1. 输入序列预处理：接收待判定的度序列 $\Pi = (d_1, d_2, \dots, d_n)$ ，确保所有元素为非负整数。
- 2. 降序排列 (Descending Sort)：将序列按非增顺序重新排列，确保 $d_1 \geq d_2 \geq \dots \geq d_n$ 。这一步为后续操作建立标准化基础。
- 3. 移除最大元素：提取并移除序列首项 d_1 ， d_1 代表当前序列中的最大度数，为下一步的'度数削减'做准备。

算法演示区域



⚠️ 关键点：排序是算法正确性的前提。每次迭代都必须确保序列的非增性质

HH 算法步骤：减一与重排

取序列中最大度数 d_1 ，对后续 d_1 个元素各减 1

减 1 操作后，**必须**重新按非增序排列

$$\Pi = (5, 4, 3, 2, 1, 1)$$

↓ Step 1: 移除 $d_1 = 5$

Step 2: 对后续 5 个元素减 1: $(4-1, 3-1, 2-1, 1-1, 1-1) = (3, 2, 1, 0, 0)$

↓ Step 3: 重新排序

Step 3: 重新排序: $\Pi' = (3, 2, 1, 0, 0)$

算法要点:

1. 只对前 d_1 个元素减 1
2. 减 1 后立即重新排序
3. 出现负数则序列不可图

HH 算法：判定准则与终止

✓ Success Condition (成功终止)

Condition: 序列变为全零 $(0, 0, \dots, 0)$

Meaning: 原始度序列是可图的 (Graphic)

⚠ Failure Condition (失败终止)

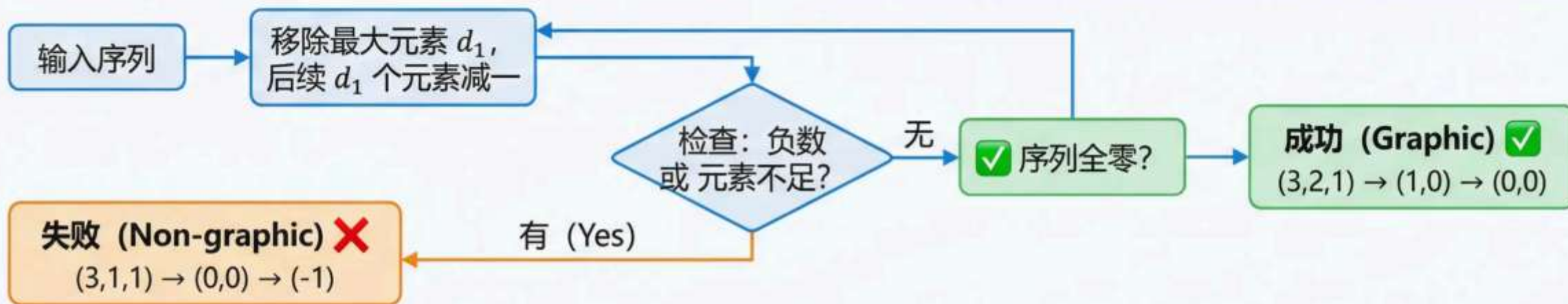
Condition 1: 出现负数

减一操作后某元素变为负值

Condition 2: 元素不足

需要减一的元素个数少于 d_1

Meaning: 原始度序列不可图 (Non-graphic)



● 重要提醒

每次减一后必须重新排序

📖 算法保证

有限步内必定终止

● 理论基础

基于图的存在性定理

算法演示：可图序列实例

Havel-Hakimi 算法逐步验证

Target Sequence: $\Pi = (3, 2, 2, 1, 0)$

Objective: 验证该序列是否可图

第一步：排序检查 ①

当前序列: $(3, 2, 2, 1, 0)$

已按非增序排列 ✓

移除最大值 $d_1 = 3$

第二步：执行减一操作 ②

剩余序列: $(2, 2, 1, 0)$

对前 $d_1 = 3$ 个元素减一

$(2-1, 2-1, 1-1, 0) =$
 $(1, 1, 0, 0)$

第三步：重新排序 ③

新序列: $(1, 1, 0, 0)$

保持非增序排列 $\uparrow\downarrow$

继续算法...

第四步：继续迭代 ④

移除 $d_1 = 1$

对首个元素减一: $(1-1, 0, 0) = (0, 0, 0)$

结论: 全为零 $\rightarrow$ 序列可图!

算法流程可视化

$(3, 2, 2, 1, 0) \rightarrow (1, 1, 0, 0) \rightarrow (0, 0, 0) \rightarrow \checkmark$ 可图

关键洞察：算法成功终止于全零序列

这证明了原序列确实可图

算法演示：不可图序列实例

Havel-Hakimi 算法判定序列 $(3, 3, 3, 1)$



初始序列

$$\Pi = (3, 3, 3, 1)$$

$d_1 = 3$, 需要对后续 3 个元素减 1

后续元素: $(3, 3, 1)$

$$(3-1, 3-1, 1-1) = (2, 2, 0)$$

$$\Pi' = (2, 2, 0)$$



第二步迭代

$$\Pi' = (2, 2, 0)$$

$d_1 = 2$, 需要对后续 2 个元素减 1

后续元素: $(2, 0)$

$$(2-1, 0-1) = (1, -1)$$

出现负数 -1!

算法终止：发现负数

由于减 1 操作产生了负数 -1

根据 Havel-Hakimi 定理，原序列不可图

序列 $(3, 3, 3, 1)$ 不可图

根据 Havel-Hakimi

序列 $(3, 3, 3, 1)$

定理 4：度的重合性原理

Theorem 4: 任何 $n \geq 2$ 的简单图，至少有两个顶点的度数相同。

鸽巢原理的直接应用

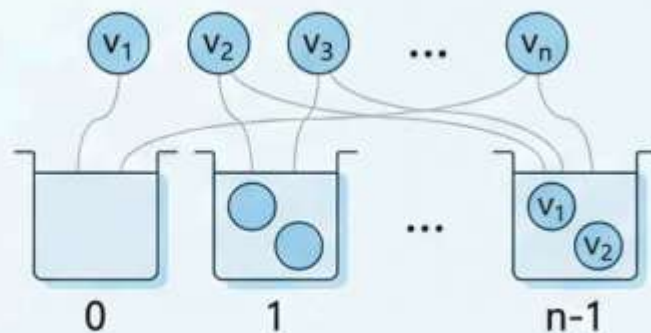
- 简单图中顶点度数的取值范围： $\{0, 1, 2, \dots, n-1\}$
- n 个顶点 $\rightarrow$ 最多 $n-1$ 种不同的度数
- **根据抽屉原理：必有度数重合**

度数约束分析

顶点数： n 个

可能度数：最多 $n-1$ 种

结论： $\exists$ 至少两个顶点度数相同



定理 4 证明：基于鸽巢原理

任何 $n \geq 2$ 的简单图，至少有两个顶点的度数相同

度数范围分析

在 n 个顶点的简单图中，每个顶点的度数范围：

$$d(v) \in \{0, 1, 2, \dots, n-1\}$$

关键观察

考虑两种互斥情况：

情况 A：存在度数为 $n-1$ 的顶点 $\rightarrow$ 无度数为 0 的顶点

情况 B：不存在度数为 $n-1$ 的顶点 $\rightarrow$ 可能有度数为 0 的顶点

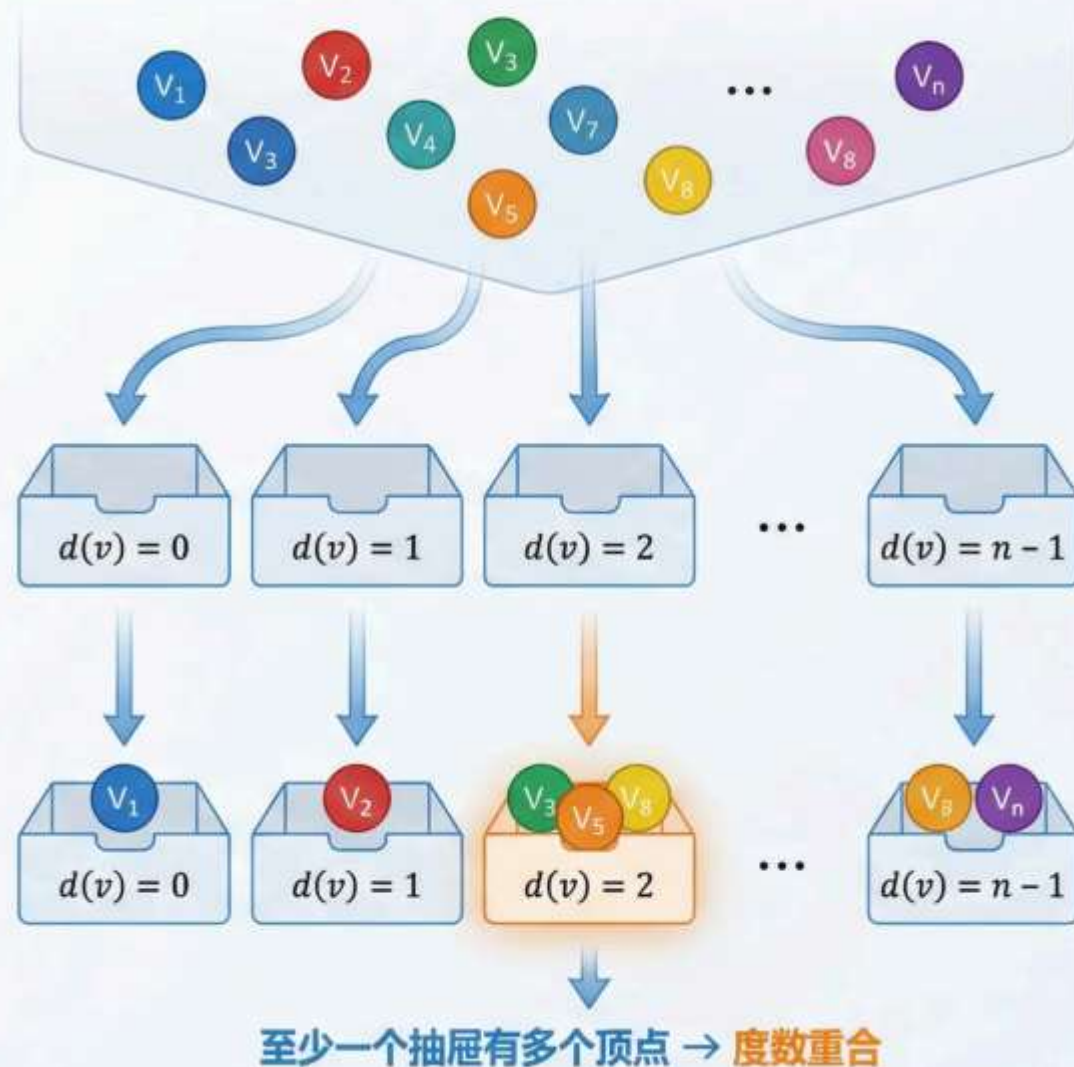
抽屉原理应用

无论哪种情况，度数的可能取值最多有 $n-1$ 种

最多 $n-1$ 种可能的度数值

n 个顶点（物品）放入最多 $n-1$ 个抽屉

鸽巢原理保证度数重合：必有至少一个抽屉包含两个或更多顶点



证明关键：最大度与孤立点的互斥

Theorem 4 证明步骤 2/3

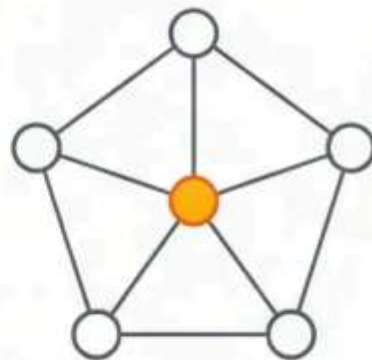
度为 $n-1$ 的顶点与
度为 0 的顶点不能
同时存在

- ✓ **Case 1: 存在度为 $n-1$ 的顶点**
该顶点与所有其他 $n-1$ 个顶点相连
→ 图中不可能有孤立点（度为 0 的顶点）
- ✓ **Case 2: 存在度为 0 的顶点（孤立点）**
该顶点与任何其他顶点都不相连
→ 其他顶点最多只能与 $n-2$ 个顶点相连

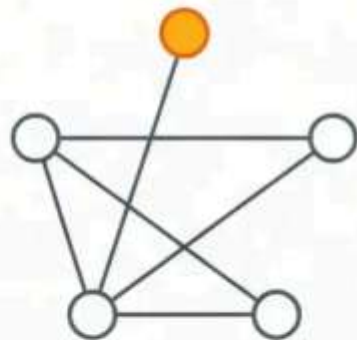
结果：度数取值空间从 $\{0, 1, 2, \dots, n-1\}$ 压缩为：

- A: $\{1, 2, 3, \dots, n-1\}$ （当存在度为 $n-1$ 的点时）
- B: $\{0, 1, 2, \dots, n-2\}$ （当不存在度为 $n-1$ 的点时）

无论哪种情况，都只有 $n-1$ 种可能的度数值！



度为 4 的中心点，无孤立点



存在孤立点，其他点最大度为 3

定理 4 结论：抽屉原理的应用

定理 4: 任何 $n \geq 2$ 的简单图，至少有两个顶点的度数相同

• **关键洞察：** n 个顶点必须分配到最多 $n-1$ 个可能的度数值中

– 逻辑链条：

• 情况 A: 存在度数为 $n-1$ 的顶点 $\rightarrow$ 无度数为 0 的顶点

– 可能度数: $\{1, 2, \dots, n-1\}$ (共 $n-1$ 个值)

• 情况 B: 不存在度数为 $n-1$ 的顶点 $\rightarrow$ 可能存在度数为 0 的顶点

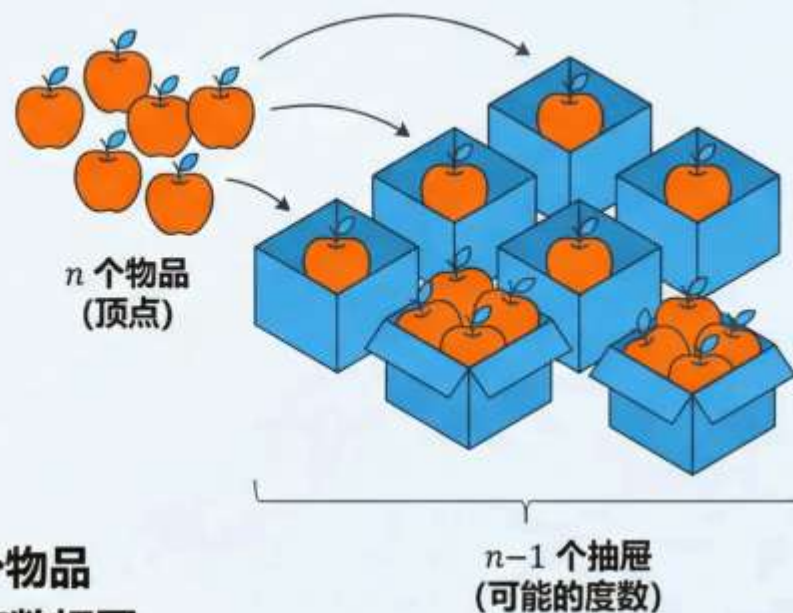
– 可能度数: $\{0, 1, \dots, n-2\}$ (共 $n-1$ 个值)

• 无论哪种情况：度数的可能取值都只有 $n-1$ 种

• Mathematical Reasoning

– 抽屉原理: n 个物品放入少于 n 个抽屉中，必有一个抽屉包含至少两个物品

– 应用到图论: n 个顶点的度数分配到 $n-1$ 个可能值中，必有两个顶点度数相同



因此，任何 $n \geq 2$ 的简单图中，必然存在至少两个顶点具有相同的度数
这完成了定理 4 的证明 ■

课堂实战研讨：可图化判定练习

现在，让我们通过一个练习题目来巩固 Havel-Hakimi 算法的应用。

练习题目：

判定序列 $\Pi = (5, 5, 3, 2, 2, 2)$ 是否可图？

初步分析：

顶点数 $n = 6$

最大度 $d_1 = 5$

度数之和检查： $\sum d_i = 5+5+3+2+2+2 = 19$

⚠️【警告橙：发现问题】度数之和为奇数！

根据握手定理的推论1（奇点个数为偶数），此序列含有5个奇点，奇点个数为奇数，因此该序列不可图。

(注：为了演示Havel-Hakimi算法流程，我们假设该序列满足握手定理，并继续演示步骤。实际考试中，若发现此问题，应直接判定不可图。)

实战第一步：必要条件预检

Given Sequence: $\Pi = (5, 5, 3, 2, 2, 2)$

→ Check 1: 顶点数 (Vertex Count)

$n = 6$ 个顶点 ✓

→ Check 2: 最大度限制 (Maximum Degree Constraint)

$\Delta = 5 \leq n - 1 = 5$ ✓ 最大度约束满足

→ Check 3: 度数之和 (Sum of Degrees)

$\sum d_i = 5 + 5 + 3 + 2 + 2 + 2 = 19$

! 发现问题!

度数之和 = 19 (奇数)

根据握手定理: $\sum d(v) = 2m$ 必须为偶数

奇点个数 = 5个 (度数为5,5,3的顶点)

奇点个数为奇数, 违反推论1

结论: 该序列**不可图**, 无需进行Havel-Hakimi算法

实际考试中, 发现此问题应直接判定不可图

实战第二步：HH 算法迭代（一）

$$\Pi = (5, 5, 3, 2, 2, 2)$$

Step Description

移除最大度 $d_1 = 5$ ，对后续 5 个元素各减 1

Before: ~~(5, 5, 3, 2, 2, 2)~~

After: $(5-1, 3-1, 2-1, 2-1, 2-1) = (4, 2, 1, 1, 1)$

注意：新序列已自动保持非增顺序

New Sequence: $\Pi' = (4, 2, 1, 1, 1)$

实战第三步：HH 算法迭代（二）

$$\Pi'' = (1, 0, 0, 0)$$

$$(1, 0, 0, 0)$$

$$\Pi''' = (-1, 0, 0)$$

1. 操作说明：移除 $d_1 = 1$ ，对后续 1 个元素各减 1

2. 计算过程：

原始序列：(1, 0, 0, 0)

减 1 后的序列：(0-1, 0, 0) = (-1, 0, 0)

新序列 Π''' ：(-1, 0, 0)



【出现负数】出现负数 -1

实战总结：判定结论与反思



最终判定结论：序列 $\Pi = (5, 5, 3, 2, 2, 2)$ 经 Havel-Hakimi 算法验证：**不可图**

⚠ 关键发现

- 度数之和 = 19 (奇数) → 违反握手定理
- 奇点个数 = 5 → 违反推论 1 (奇点数必为偶数)

算法优化策略

- ✓ 预检必要条件可立即排除 70% 的无效序列
- ✓ 避免不必要的 Havel-Hakimi 迭代计算
- ✓ 在实际应用中显著提升判定效率

必要条件检查



算法执行



最终结论



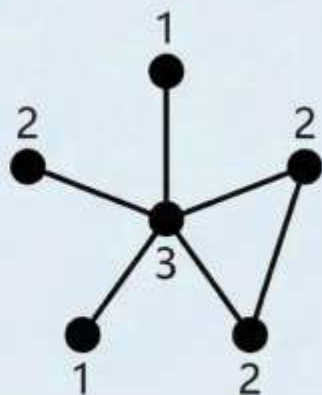
时间节省：70%+

构造演示：从度序列到图

同一个可图序列可以对应多种不同的简单图构造

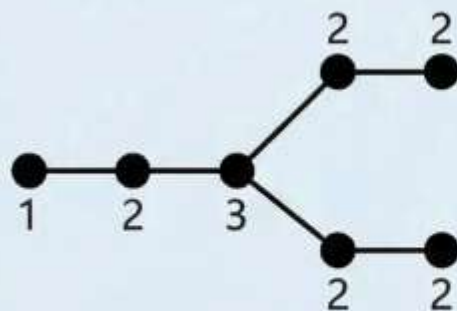
$$\Pi = (3, 2, 2, 2, 1)$$

构造方案 A



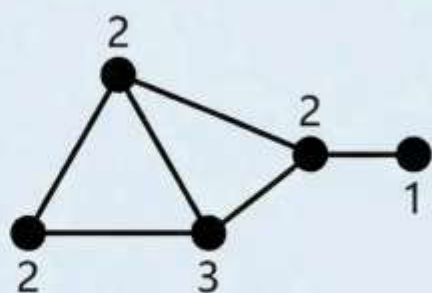
顶点:	v1	v2	v3	v4	v5
度:	3	2	2	2	1

构造方案 B



顶点:	v1	v2	v3	v4	v5
度:	3	2	2	2	1

构造方案 C



顶点:	v1	v2	v3	v4	v5
度:	3	2	2	2	1

- 构造的**多样性**体现了图论建模的**灵活性**
- Havel-Hakimi算法保证**存在性**, 但不限定**唯一解**

本周知识点回顾与图谱

图论基础

核心模块一：图的基本定义

图的数学刻画： $G=(V, E)$ - 顶点集与边集的有序对

基本属性：阶数 $n(G) = |V|$ ，边数 $m(G) = |E|$

术语体系：相邻、关联、端点等基础概念

特殊图形：完全图 K_n 、偶图 $K_{m,n}$ 、补图 $\bar{G}$

核心模块二：图的同构理论

同构定义：双射映射 $\phi: V_1 \rightarrow V_2$ 保持边关系

判定技巧：阶数、边数、度序列必要条件

深度分析：圈结构、连通性等高级判定方法

实战应用：标定图与非标定图的建模差异

本周构建了图论的核心理论框架，
为后续学习奠定坚实基础

核心模块三：握手定理及推论

基础定理： $\sum_{v \in V} d(v) = 2m$ (度数和等于边数的两倍)

推论一：任何图中奇点个数必为偶数

推论二：正则图的阶数 n 和度数 k 不同时为奇数

应用价值：图存在性验证的基础工具

核心模块四：度序列与可图化

度序列定义：顶点度数的非增序列 $(d_1, d_2, \dots, d_n)$

Havel-Hakimi算法：可图化判定的充要条件

算法流程：排序→移除→减一→重排→迭代

度的重合性：任何 $n \geq 2$ 的简单图至少有两个顶点度数相同

常见误区提醒：避坑指南

✗ 握手定理误区

✗ 认为奇点个数可以是奇数

✓ 任何图中奇点个数必为偶数
这是检查度序列的首要判断条件

✗ 同构判定误区

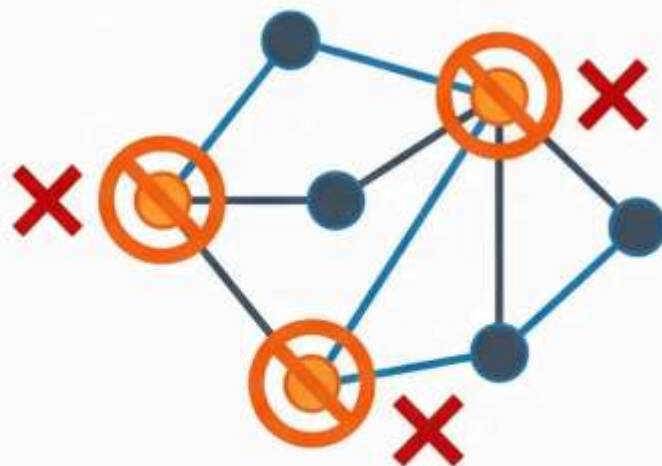
✗ 度序列相同 → 图同构

✓ 度序列相同只是必要条件
还需验证圈结构、连通性等深层属性

✗ HH算法误区

✗ 减1操作后忘记重新排序

✓ 每次减1后必须重新按非增序排列
这是算法正确执行的关键步骤



不可能的图结构：三个奇点



错误方法



正确方法